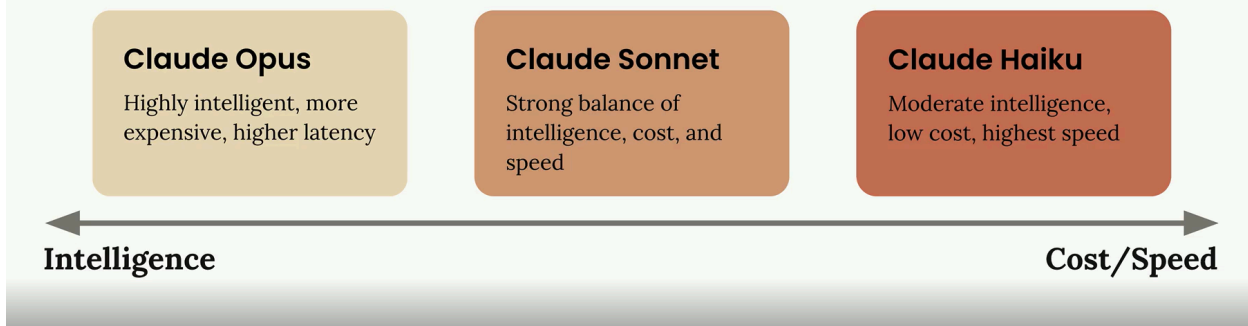


Claude Models:

Claude model family

	Claude Opus	Claude Sonnet	Claude Haiku
Description	Highest level of intelligence	Intelligent model that balances quality, speed, cost	Most cost-efficient and latency-optimized model
Cost	High	Medium	Low
Comparative latency	Moderate	Fast	Fastest
Supports reasoning	Yes	Yes	No
Best used for	• Advanced software development, especially large-scale architecting • Long running tasks that require sustained focus • Strategic planning with multi-step problem solving • Tasks that could benefit from advanced reasoning	• Common coding tasks • Document creation and editing • Content marketing and copywriting • Data analysis and visualization projects • Image analysis • Process automation	• Quick code completions and suggestions • Content moderation and filtering • Data extraction and categorization • Language translation • Q&A systems and knowledge retrieval • Most high-volume, straightforward text processing tasks

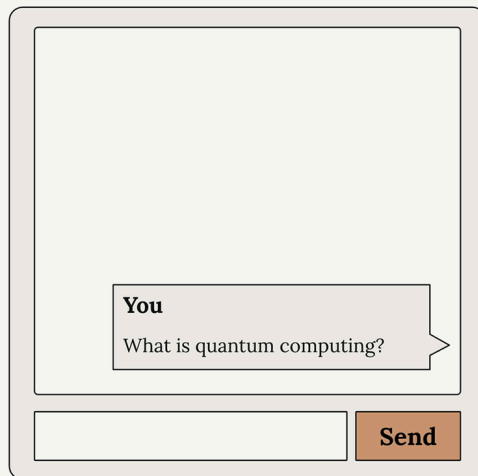
Picking the right model



Accessing API

When building applications with Claude, understanding the complete request lifecycle helps you make better architectural decisions and debug issues more effectively. Let's walk through what happens from the moment a user clicks "send" in your chat interface to when Claude's response appears on screen.

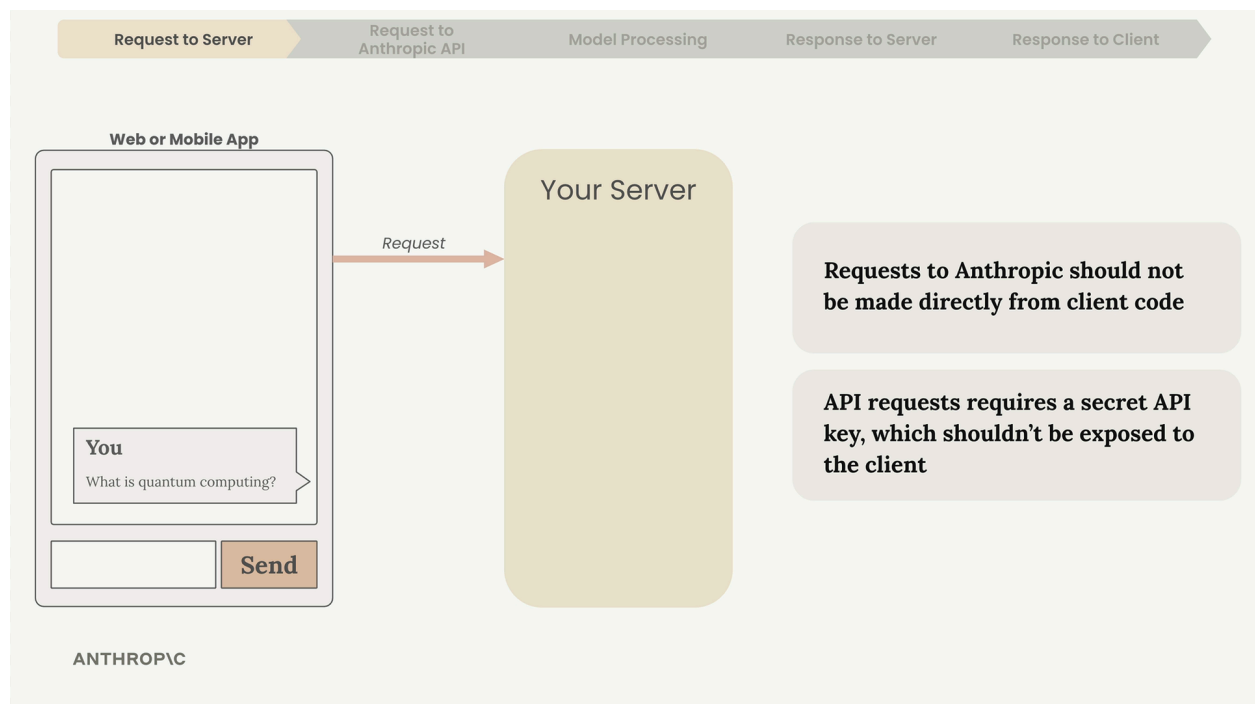
Chat example



ANTHROPIC

The Five-Step Request Flow

Every interaction with Claude follows a predictable pattern with five distinct phases: request to server, request to Anthropic API, model processing, response to server, and response to client.



Why You Need a Server

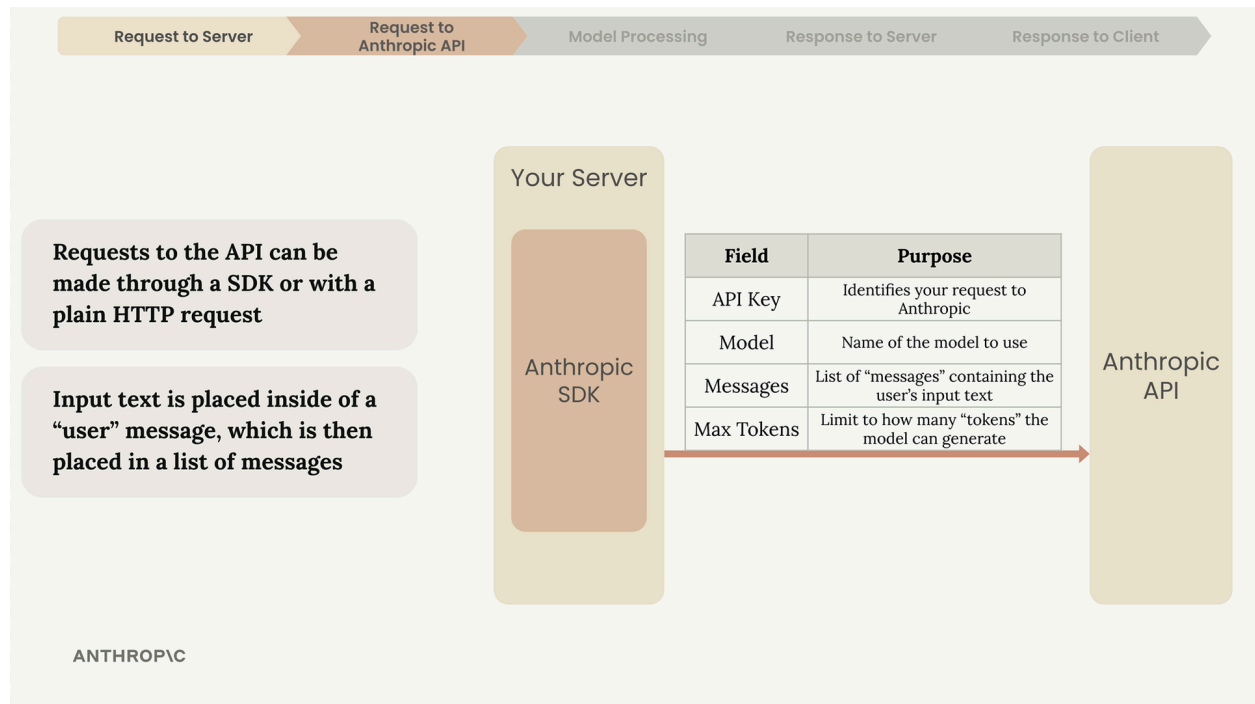
You should never make requests to the Anthropic API directly from client-side code. Here's why:

- API requests require a secret API key for authentication
- Exposing this key in client code creates a serious security vulnerability
- Anyone could extract the key and make unauthorized requests

Instead, your web or mobile app sends requests to your own server, which then communicates with the Anthropic API using the securely stored key.

Making API Requests

When your server contacts the Anthropic API, you can use either an official SDK or make plain HTTP requests. Anthropic provides SDKs for Python, TypeScript, JavaScript, Go, and Ruby.

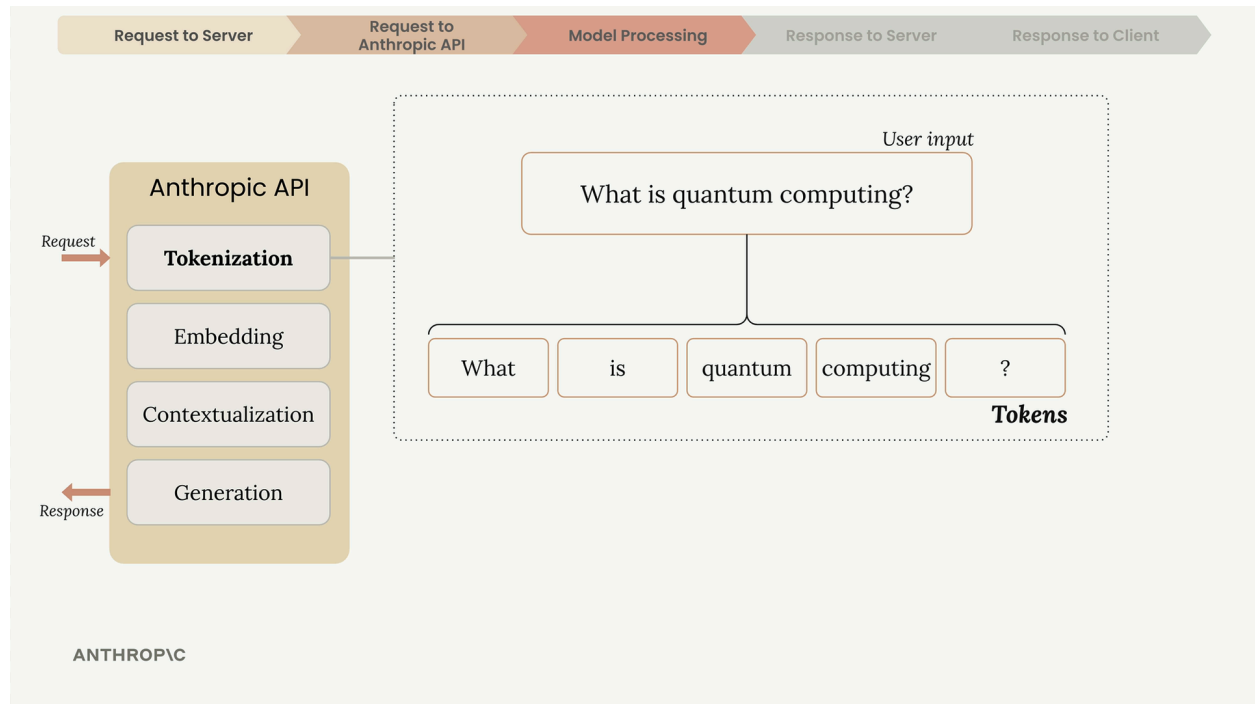


Every request must include these essential fields:

- **API Key** - Identifies your request to Anthropic
- **Model** - Name of the model to use (like "claude-3-sonnet")
- **Messages** - List containing the user's input text
- **Max Tokens** - Limit for how many tokens Claude can generate

Inside Claude's Processing

Once Anthropic receives your request, Claude processes it through four main stages: tokenization, embedding, contextualization, and generation.

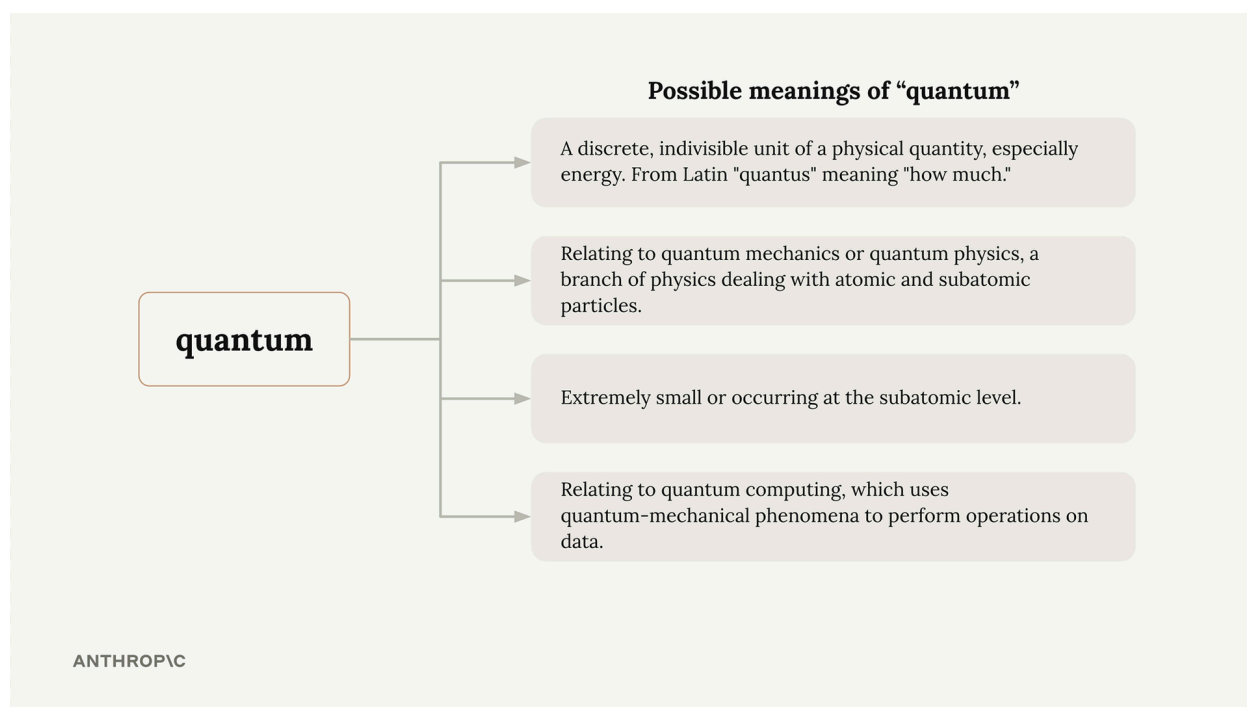


Tokenization

Claude first breaks your input text into smaller chunks called tokens. These can be whole words, parts of words, spaces, or symbols. For simplicity, think of each word as one token.

Embedding

Each token gets converted into an embedding - a long list of numbers that represents all possible meanings of that word. Think of embeddings as numerical definitions that capture semantic relationships.

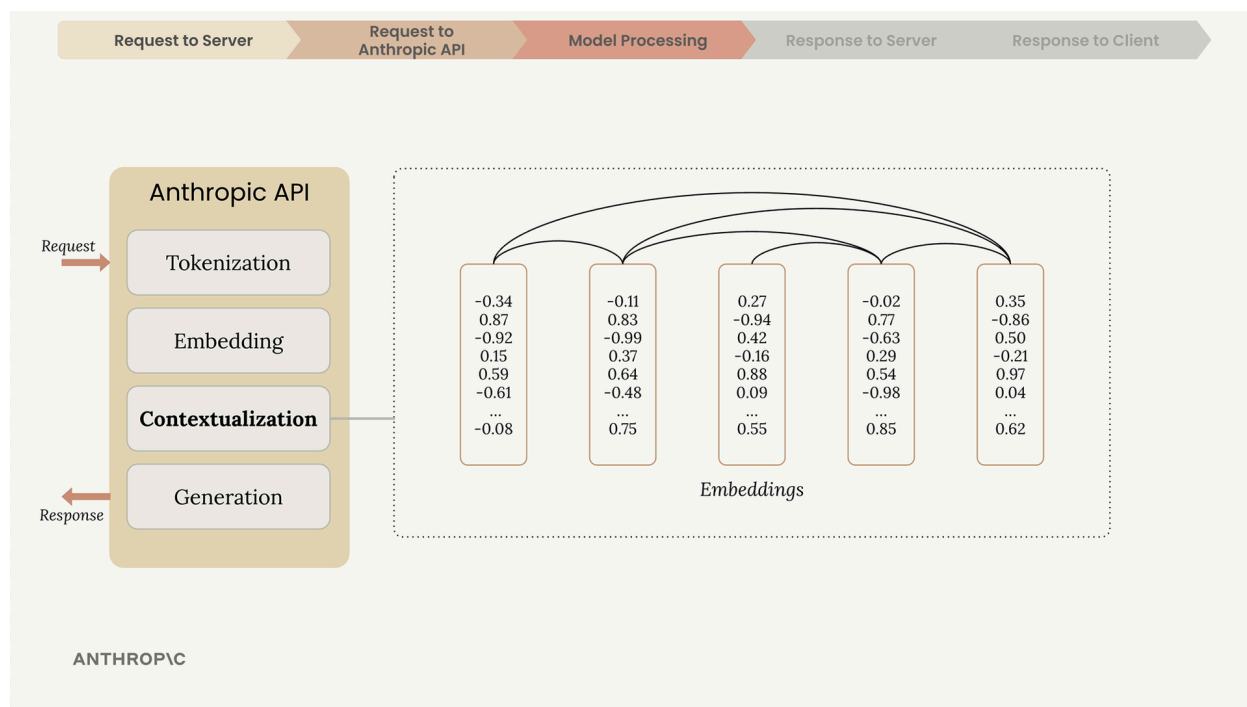


Words often have multiple meanings. For example, "quantum" could refer to:

- A discrete unit of physical quantity (physics)
- Quantum mechanics or quantum physics concepts
- Something extremely small or subatomic
- Quantum computing applications

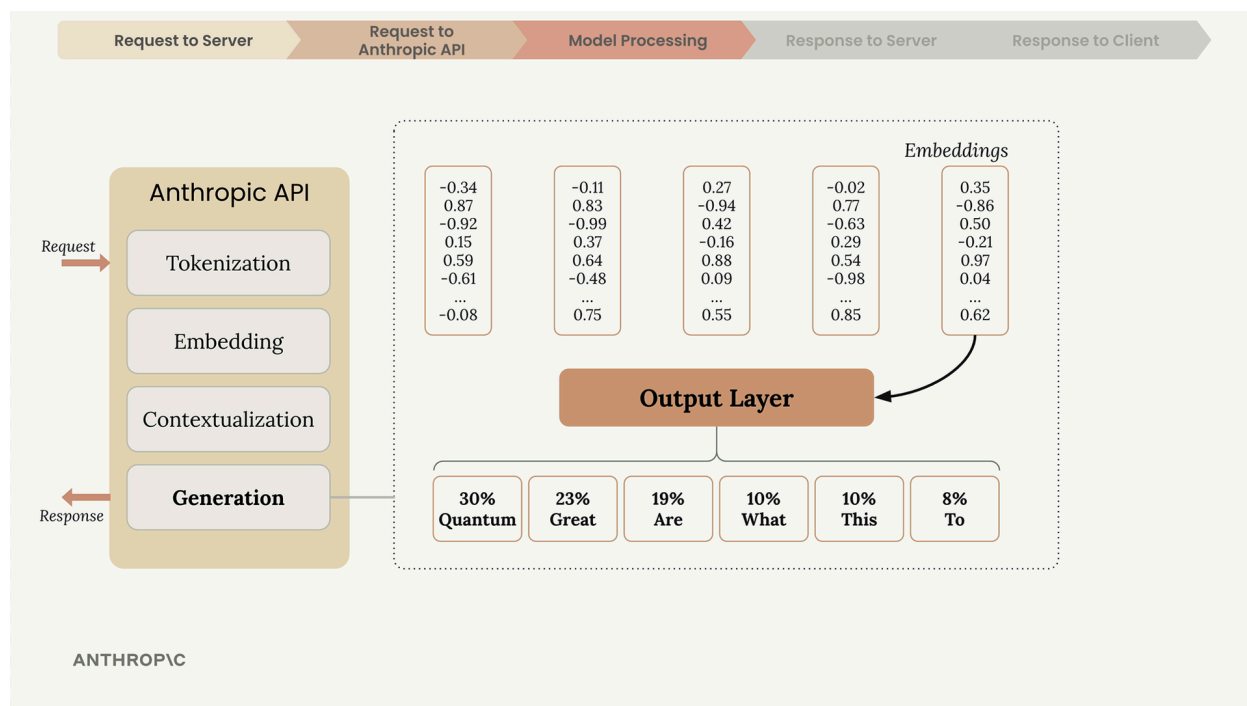
Contextualization

Claude refines each embedding based on surrounding words to determine the most likely meaning in context. This process adjusts the numerical representations to highlight the appropriate definition.



Generation

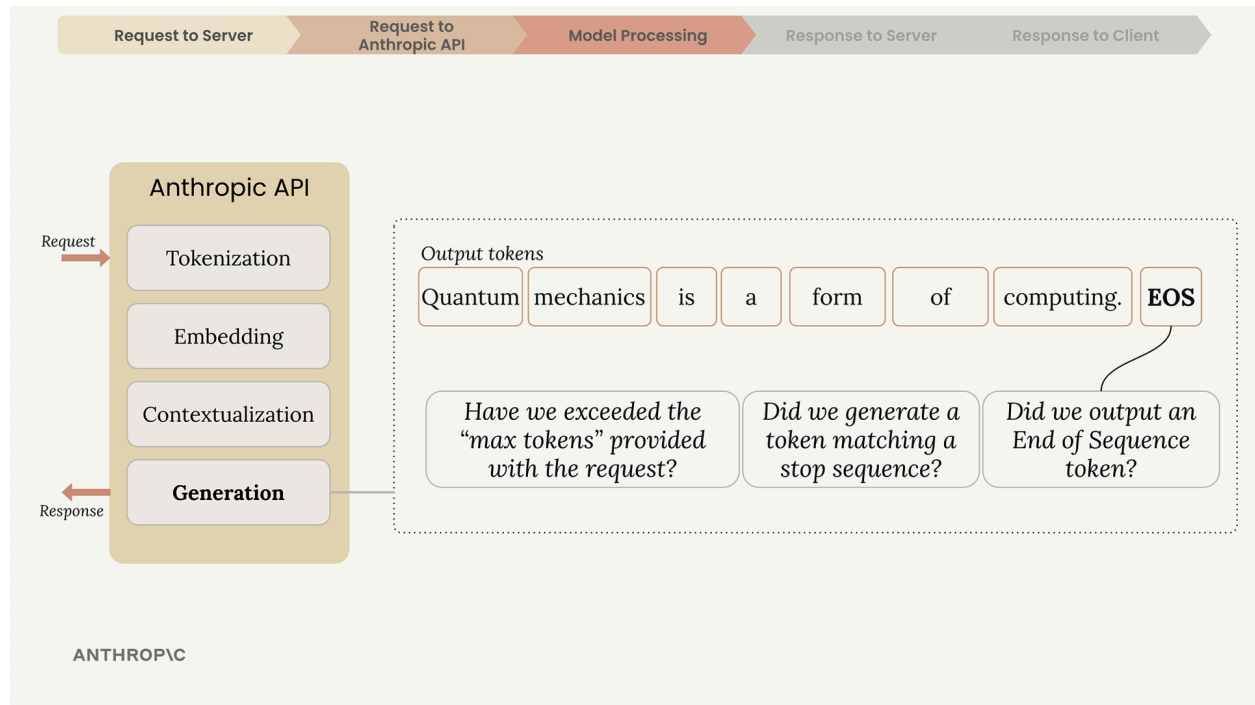
The contextualized embeddings pass through an output layer that calculates probabilities for each possible next word. Claude doesn't always pick the highest probability word - it uses a mix of probability and controlled randomness to create natural, varied responses.



After selecting each word, Claude adds it to the sequence and repeats the entire process for the next word.

When Claude Stops Generating

After each token, Claude checks several conditions to decide whether to continue:

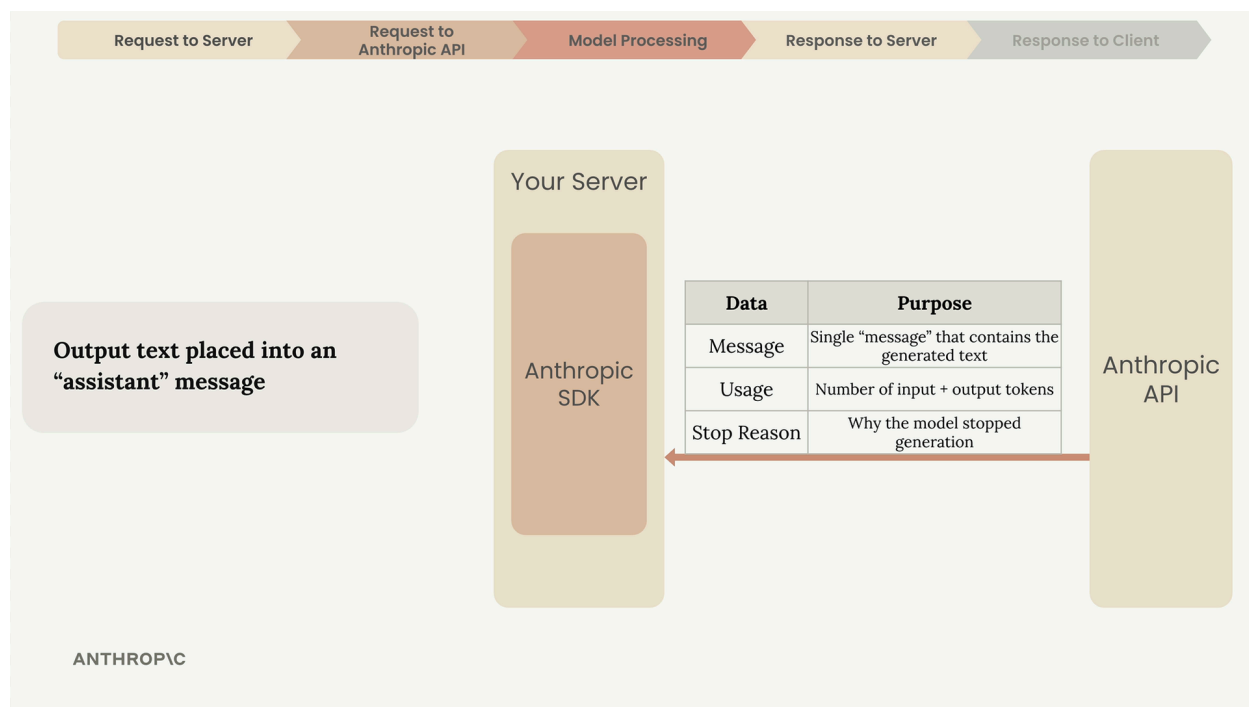


- **Max tokens reached** - Has it hit the limit you specified?
- **Natural ending** - Did it generate an end-of-sequence token?
- **Stop sequence** - Did it encounter a predefined stop phrase?

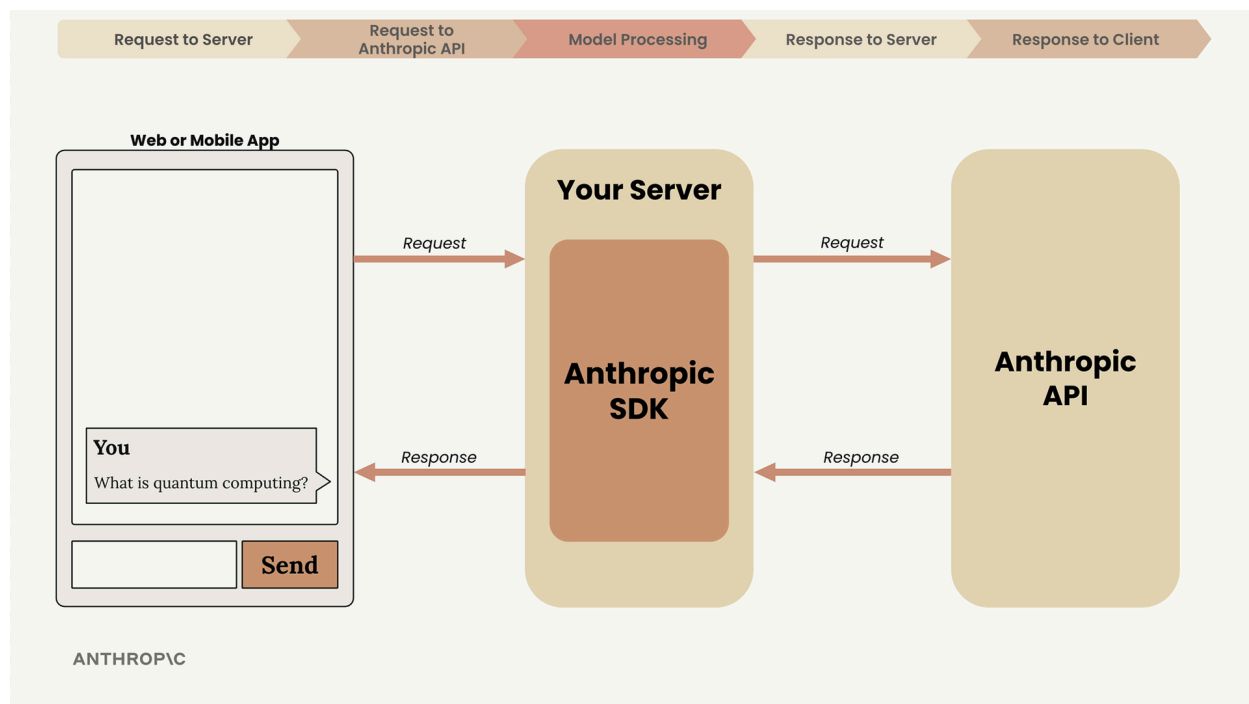
The API Response

When generation completes, the API sends back a structured response containing:

- **Message** - The generated text
- **Usage** - Count of input and output tokens
- **Stop Reason** - Why generation ended



Your server receives this response and forwards the generated text back to your client application, where it appears in the user interface.



Key Takeaways

Understanding this flow helps you:

- Design secure architectures that protect your API keys
- Set appropriate token limits for your use case
- Handle different stop reasons in your application logic
- Debug issues by understanding where they might occur in the pipeline

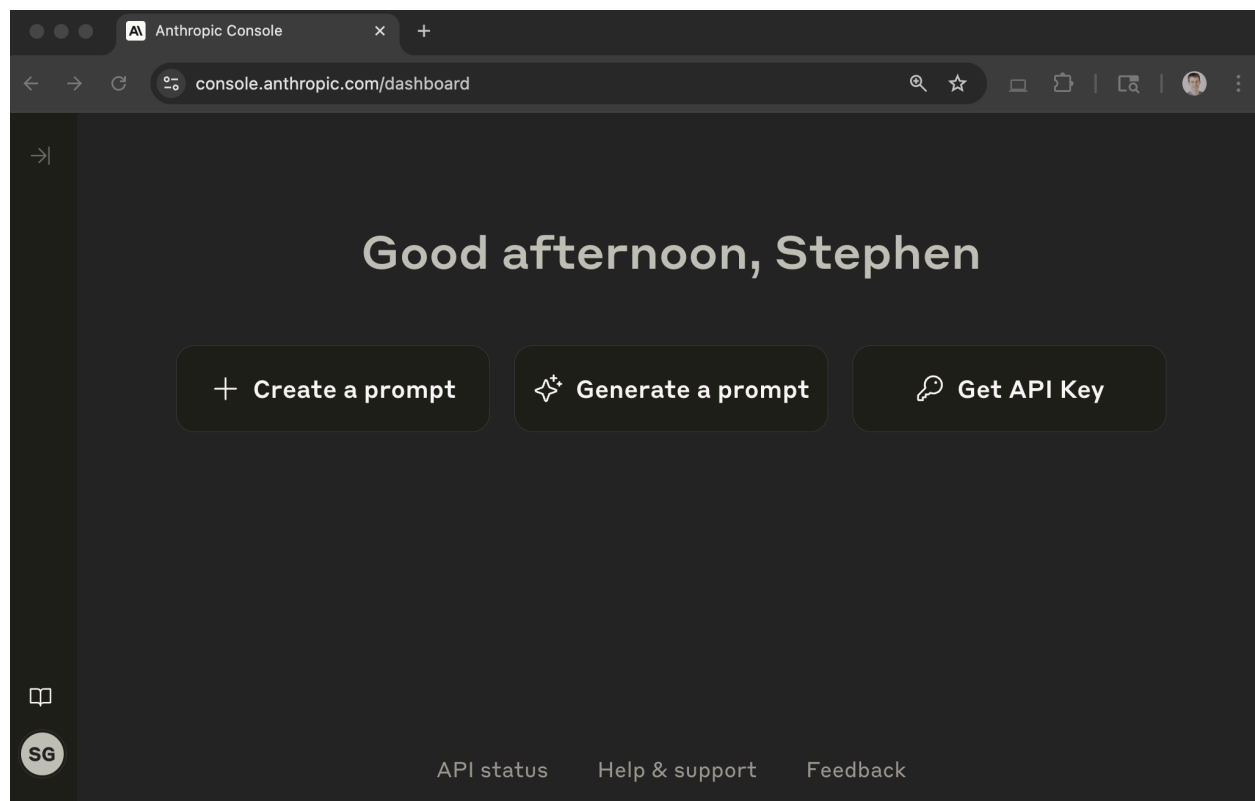
Don't worry about memorizing every detail - the goal is familiarizing yourself with the terminology and overall process you'll encounter when working with Claude's API.

Getting an API key

In the next video we will be making a request to the Anthropic API. To do so, you will need a secret API key. This guide will walk you through the process of creating an API key.

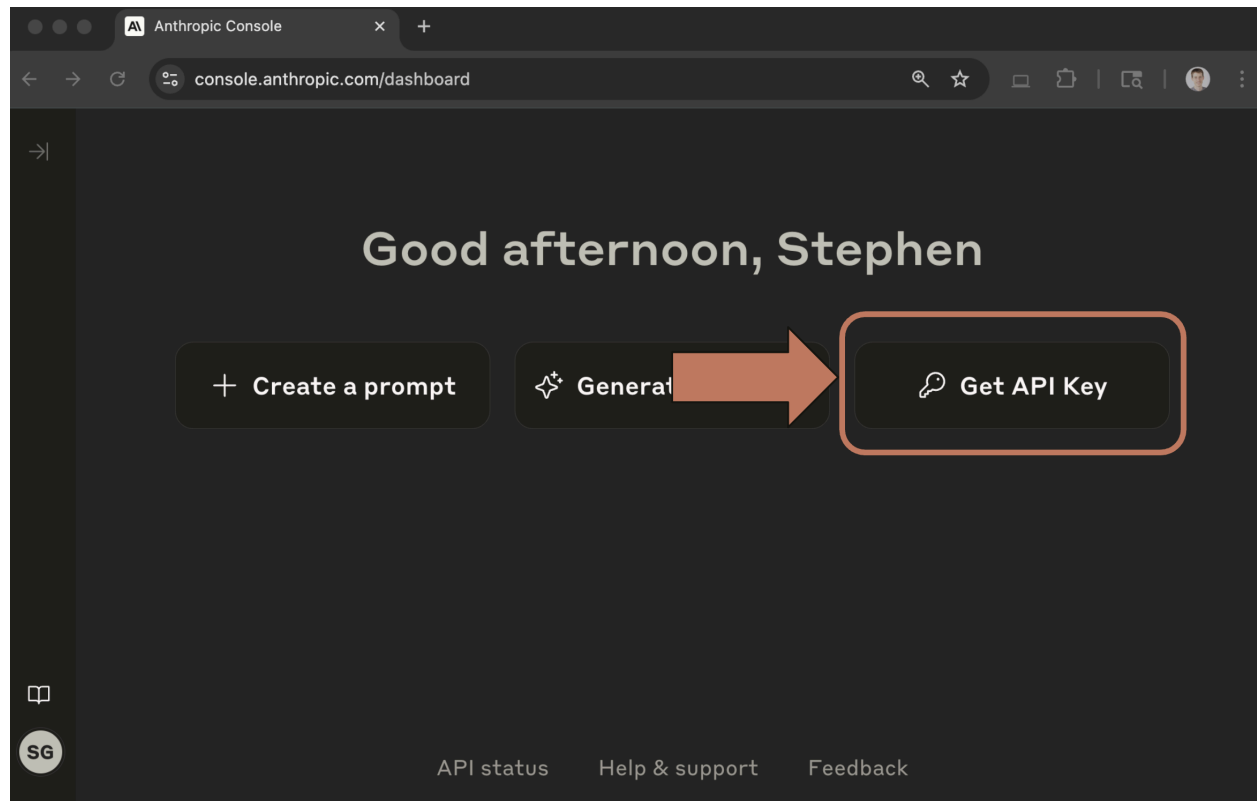
Step One: Navigate to the Anthropic API Console

In your browser, navigate to <https://console.anthropic.com/> and log in to your Anthropic account. You'll then see a page like this:



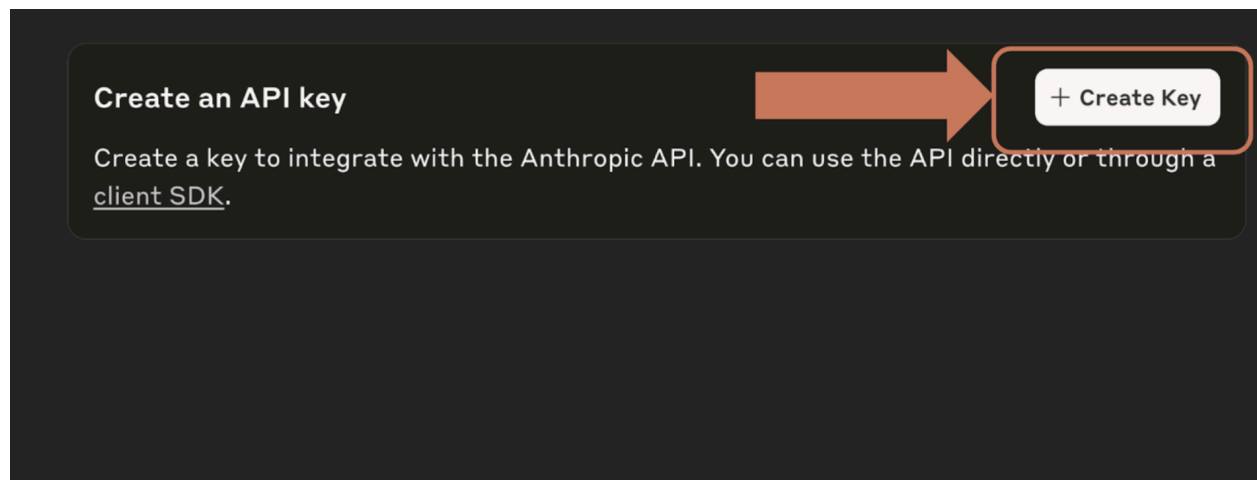
Step Two: Click the 'Get API Keys' button

This button can be found towards the top right of the main dashboard page.



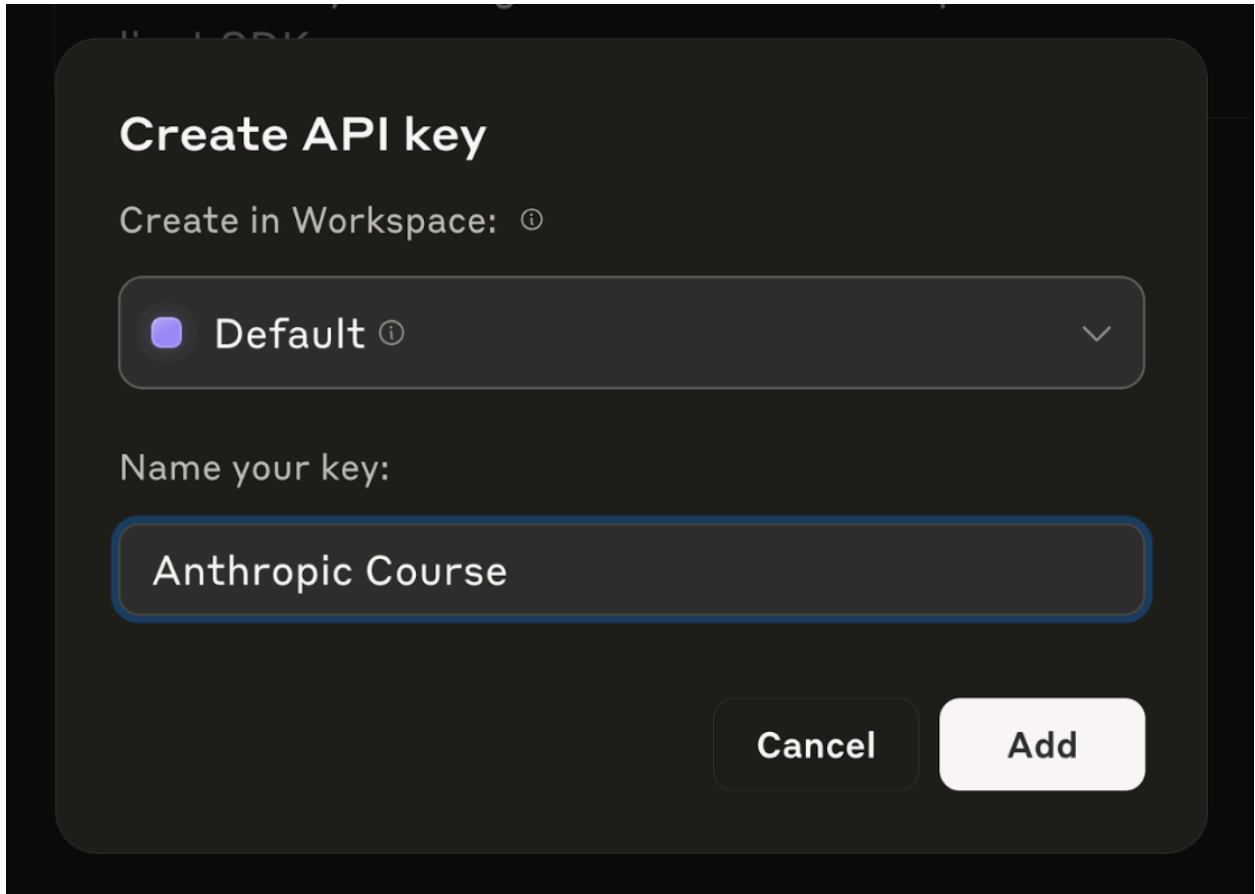
Step Three: Click the 'Create Key' button

At the top right of the page, find the 'Create Key' button and click it.



Step Four: Enter a workspace and name for your key

Create the key in workspace 'Default' and enter a name for your key. This name is used to help you identify the keys you generate. Let's use a name of 'Anthropic Course'.



Create API key

Create in Workspace: ⓘ

Default ⓘ

Name your key:

Anthropic Course

Cancel Add

Step Five: Copy the Key

Your API key will then be displayed in a pop up window. Copy this key and hold onto it - we will use it in the next video. This key will only be displayed once, so make sure you copy it!

If you accidentally close the window, delete the old key and generate it again.

Making the Request

Making your first request to the Anthropic API is straightforward once you understand the basic setup and structure. This guide walks through the essential steps to get Claude responding to your prompts using Python.

Setting Up Your Environment

Before making any API calls, you need to install the required packages and configure your API key securely.

First, install the necessary dependencies in your Jupyter notebook:

```
%pip install anthropic python-dotenv
```

Next, create a `.env` file in the same directory as your notebook to store your API key securely:

```
ANTHROPIC_API_KEY="your-api-key-here"
```

This approach keeps your API key out of your code and prevents accidentally committing it to version control. Always add `.env` to your `.gitignore` file.

Load the environment variables and create your API client:

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
from anthropic import Anthropic
```

```
client = Anthropic()
```

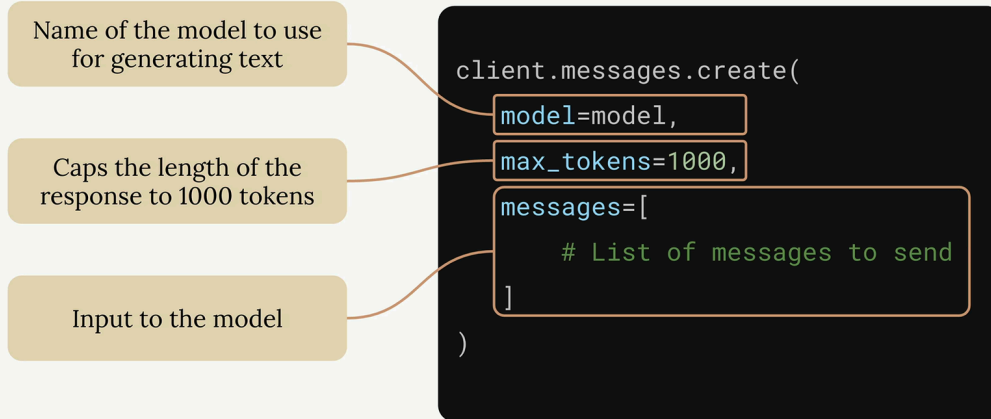
```
model = "claude-sonnet-4-0"
```

The Create Function

The core of making API requests is the `client.messages.create()` function. This function requires three key parameters:

The 'create' function

Generates text by making a request to Claude



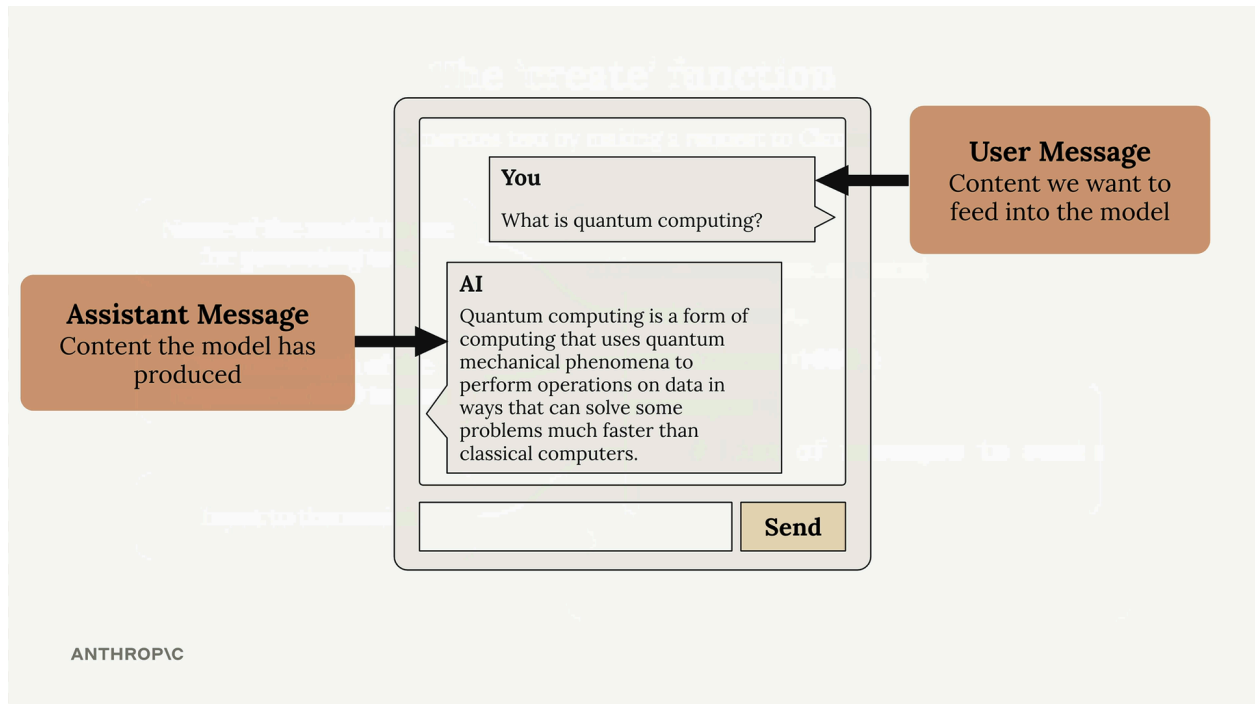
ANTHROPIC

- **model** - The name of the Claude model you want to use
- **max_tokens** - A safety limit on response length (not a target)
- **messages** - The conversation history you're sending to Claude

The `max_tokens` parameter acts as a safety mechanism. If you set it to 1000, Claude will stop generating after 1000 tokens even if it has more to say. Claude doesn't try to reach this limit - it just writes what it thinks is appropriate and stops if it hits the maximum.

Understanding Messages

Messages represent the conversation between you and Claude, similar to a chat application. There are two types of messages:



- **User messages** - Content you want to send to Claude (written by humans)
- **Assistant messages** - Responses that Claude has generated

Each message is a dictionary with a **role** (either "user" or "assistant") and **content** (the actual text).

Making Your First Request

Here's a complete example of making a request to Claude:

```
message = client.messages.create(  
    model=model,  
    max_tokens=1000,  
    messages=[  
        {  
            "role": "user",  
            "content": "What is quantum computing? Answer in one sentence"  
        }  
    ]  
)
```

]

)

When you run this code, Claude will process your request and return a response object containing the generated text along with metadata about the request.

Extracting the Response

The response object contains a lot of information, but you usually just want the generated text. Access it using:

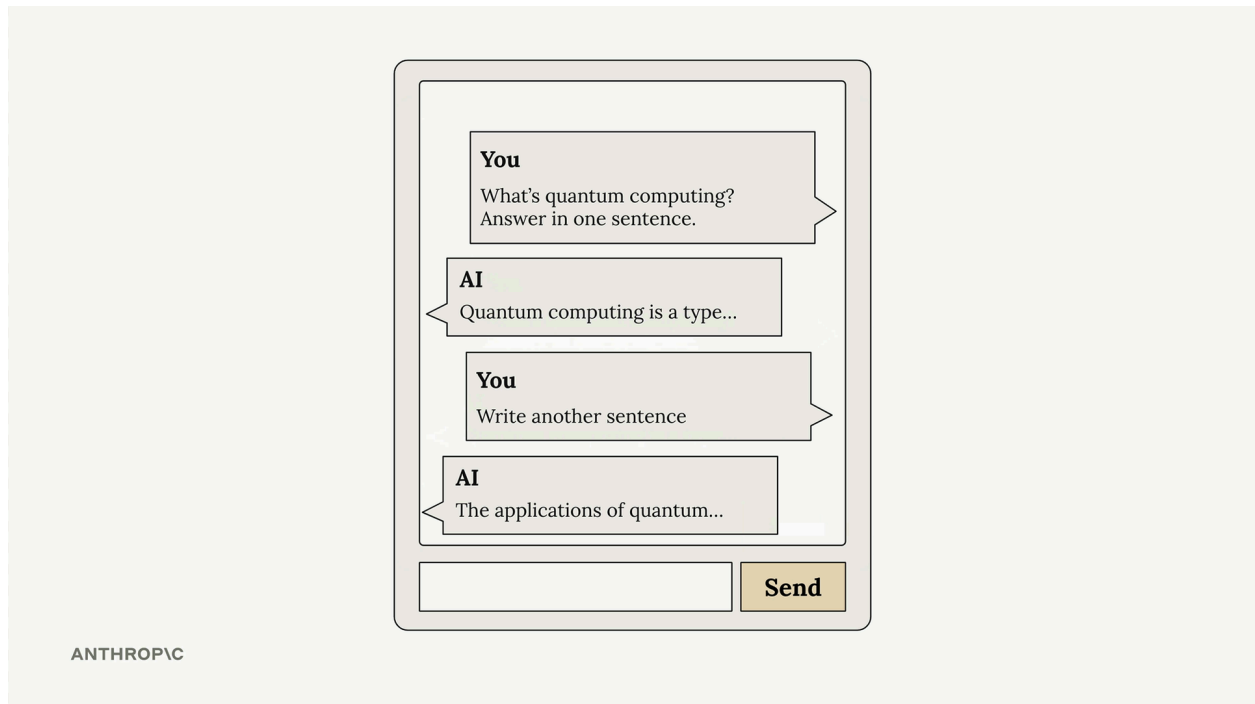
```
message.content[0].text
```

This gives you clean, readable output like: "Quantum computing is a type of computation that leverages quantum mechanics principles like superposition and entanglement to process information using quantum bits (qubits), potentially solving certain complex problems exponentially faster than classical computers."

With these basics in place, you can start experimenting with different prompts and building more complex interactions with Claude.

Multi Turn Conversations

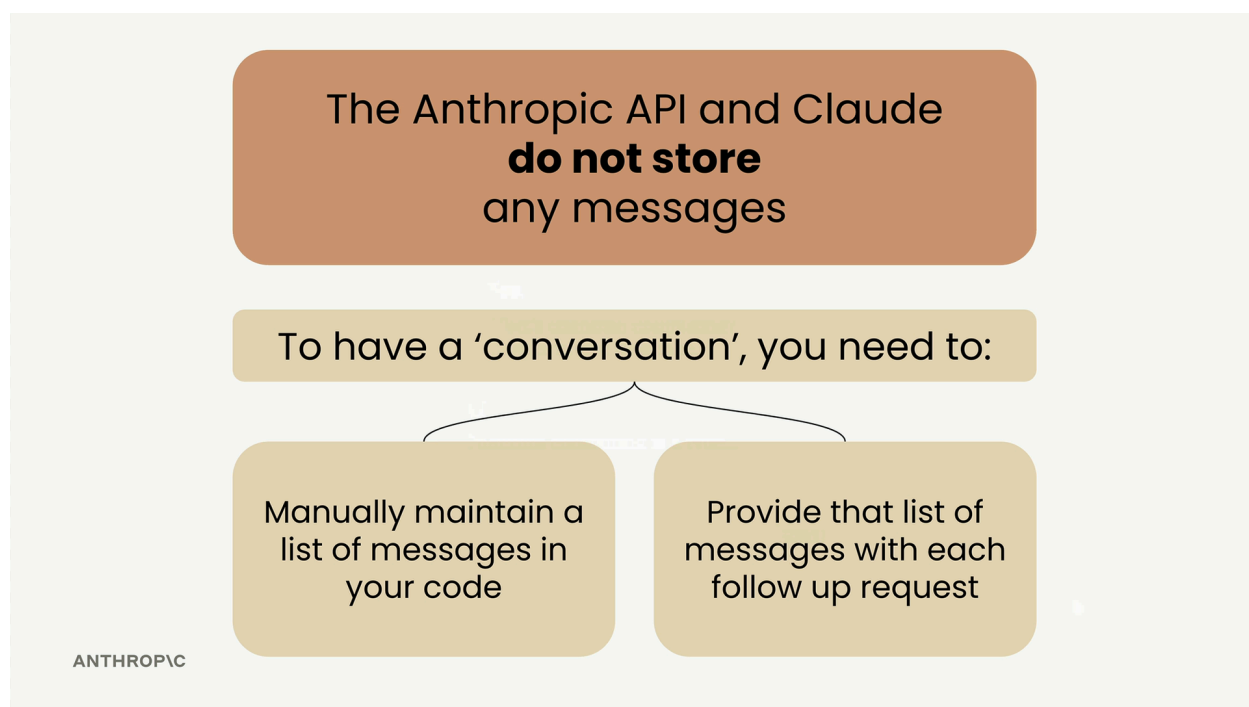
When working with the Anthropic API and Claude, there's a crucial concept you need to understand: **Claude doesn't store any of your conversation history**. Each request you make is completely independent, with no memory of previous exchanges.



This means if you want to have a multi-turn conversation where Claude remembers context from earlier messages, you need to handle the conversation state yourself.

The Problem with Stateless Conversations

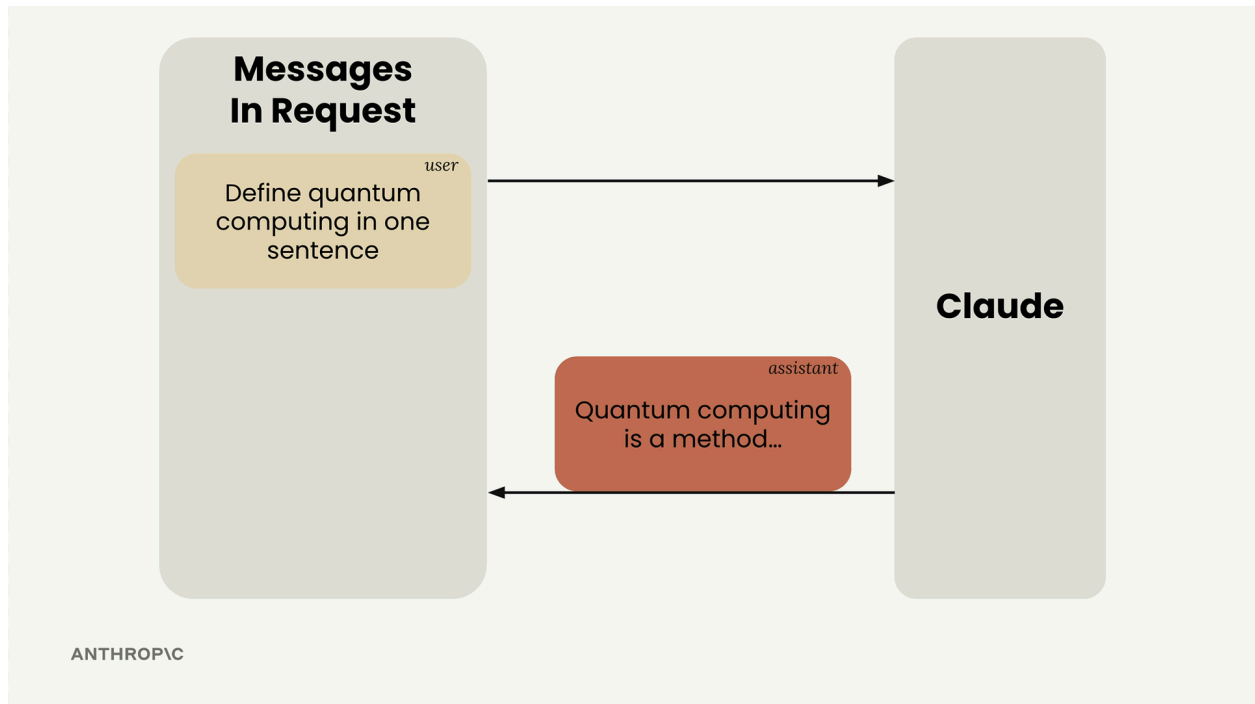
Let's say you ask Claude "What is quantum computing?" and get a good response. Then you follow up with "Write another sentence" - Claude has no idea what you're referring to. It will write a sentence about something completely random because it has no memory of the quantum computing discussion.



How Multi-Turn Conversations Work

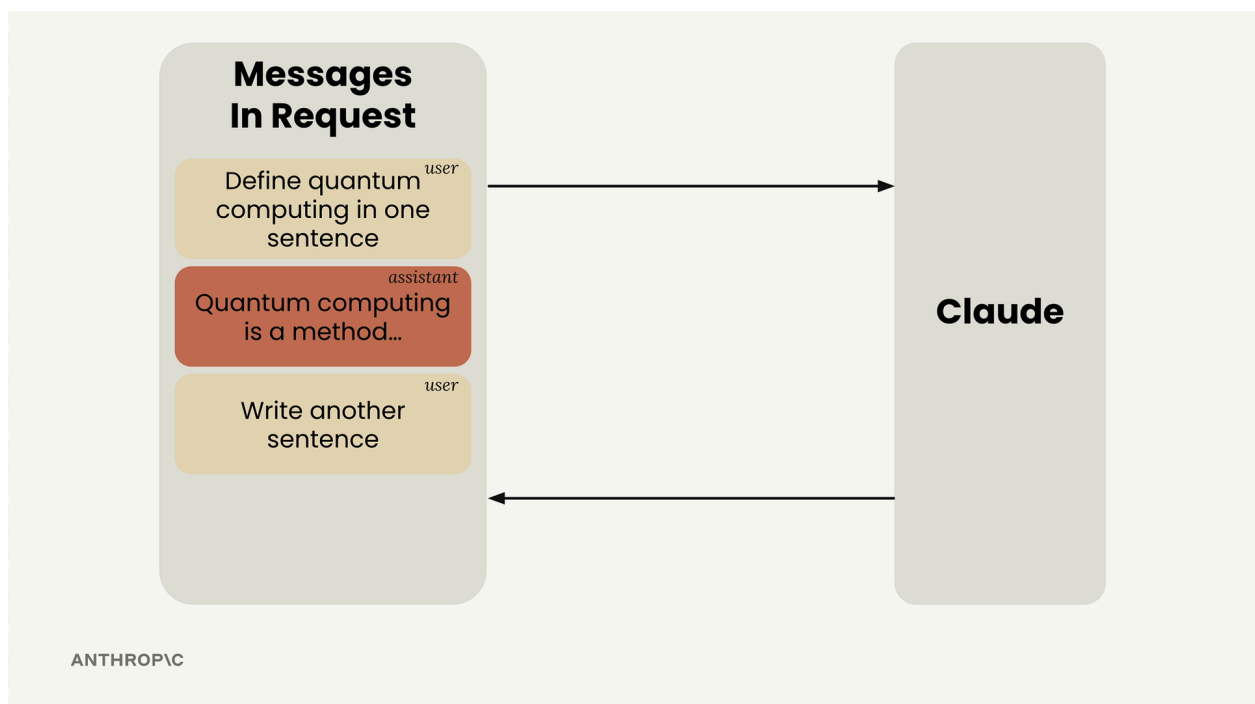
To maintain conversation context, you need to do two things:

- Manually maintain a list of all messages in your code
- Send the complete message history with every request



Here's the flow that actually works:

1. Send your initial user message to Claude
2. Take Claude's response and add it to your message list as an assistant message
3. Add your follow-up question as another user message
4. Send the entire conversation history to Claude



Building Helper Functions

To make conversation management easier, you can create three helper functions:

```
def add_user_message(messages, text):  
    user_message = {"role": "user", "content": text}  
    messages.append(user_message)  
  
def add_assistant_message(messages, text):  
    assistant_message = {"role": "assistant", "content": text}  
    messages.append(assistant_message)  
  
def chat(messages):  
    message = client.messages.create(  
        model=model,  
        max_tokens=1000,  
        messages=messages,  
    )  
    return message.content[0].text
```

Putting It All Together

Here's how you use these functions to maintain a conversation:

```
# Start with an empty message list  
messages = []  
  
# Add the initial user question  
add_user_message(messages, "Define quantum computing in one sentence")
```

```
# Get Claude's response
```

```
answer = chat(messages)
```

```
# Add Claude's response to the conversation history
```

```
add_assistant_message(messages, answer)
```

```
# Add a follow-up question
```

```
add_user_message(messages, "Write another sentence")
```

```
# Get the follow-up response with full context
```

```
final_answer = chat(messages)
```

Now Claude will understand that "Write another sentence" refers to expanding on the quantum computing definition, because you've provided the complete conversation context.

These helper functions will be useful throughout your work with Claude, making it much easier to build applications that can maintain meaningful conversations over multiple exchanges.

Chat Bot Exercise

Make a chat bot using the three helper functions we just put together.

1. Prompt the user to enter some input using the built-in "input" function
2. Add it to a list of messages
3. Call the API
4. Add generated text to the list of messages
5. Print the generated text
6. Repeat from #1

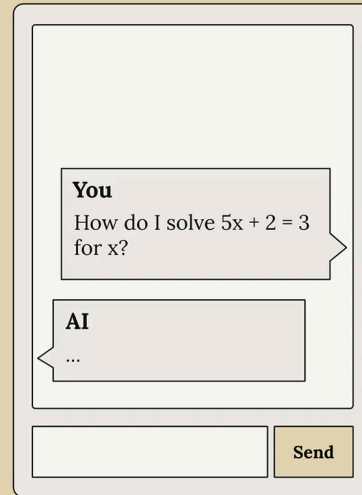
System Prompt

System prompts are a powerful way to customize how Claude responds to user input. Instead of getting generic answers, you can shape Claude's tone, style, and approach to match your specific use case.

Math Tutor Specialist

- **Responses should:**
 - Initially, only give the student hints
 - Patiently walk the student through a solution
 - Show solutions for similar problems
- **Responses should not:**
 - Immediately give a direct answer
 - Tell the student to use a calculator

ANTHROPIC



Why System Prompts Matter

Consider building a math tutor chatbot. When a student asks "How do I solve $5x + 2 = 3$ for x ?", you want Claude to act like a real tutor, not just spit out the answer. A good math tutor should:

- Initially give hints rather than complete solutions
- Patiently walk students through problems step by step
- Show solutions for similar problems as examples

You definitely don't want Claude to:

- Immediately give direct answers
- Tell students to just use a calculator

How System Prompts Work

System prompts

Customizes Claude's tone and style of response

System prompts provide Claude guidance on how to respond

Claude will try to respond in the same way someone in the specified role would respond

Helps keep Claude on task

ANTHROPIC

```
system_prompt=""  
You are a patient math tutor. Do not directly  
answer a student's questions. Guide them to a  
solution step by step.  
""  
  
client.messages.create(  
    model=model,  
    messages=messages,  
    max_tokens=1000,  
    system=system_prompt  
)
```

System prompts provide Claude with guidance on how to respond. You define them as plain strings and pass them into the create function call. The key benefits are:

- System prompts provide Claude guidance on how to respond
- Claude will try to respond in the same way someone in the specified role would respond
- Helps keep Claude on task

Here's the basic structure:

```
system_prompt = ""  
You are a patient math tutor.  
Do not directly answer a student's questions.  
Guide them to a solution step by step.  
""
```

```
client.messages.create(  
    model=model,  
    messages=messages,  
    max_tokens=1000,  
    system=system_prompt  
)
```

Seeing the Difference

Without a system prompt, Claude gives a complete step-by-step solution immediately. This might be helpful, but it doesn't encourage the student to think through the problem themselves.

With the math tutor system prompt, Claude's response changes dramatically. Instead of providing the full solution, Claude asks guiding questions like "What do you think would be a good first step to isolate x? Consider what operation we might need to perform on both sides to start moving terms around."

Building a Flexible Chat Function

Rather than hard-coding system prompts, you can make your chat function more reusable by accepting system prompts as parameters:

```
def chat(messages, system=None):
    params = {
        "model": model,
        "max_tokens": 1000,
        "messages": messages,
    }

    if system:
        params["system"] = system

    message = client.messages.create(**params)
    return message.content[0].text
```

This approach handles an important detail: Claude's API doesn't accept `system=None`, so you need to conditionally include the system parameter only when it's provided.

Now you can call your chat function with or without a system prompt:

```
# Without system prompt
answer = chat(messages)

# With system prompt
system = """
You are a patient math tutor.
Do not directly answer a student's questions.
Guide them to a solution step by step.
"""
answer = chat(messages, system=system)
```

System prompts are essential for creating AI applications that behave consistently and appropriately for their intended purpose. They transform generic AI responses into specialized, role-appropriate interactions.

```
002_system_prompt.ipynb > messages = []
Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ...
Python 3.9.6

messages = []

add_user_message(
    messages,
    "Write a Python function that checks a string for duplicate characters.",
)

answer = chat(messages)

answer

[32] ✓ 6.4s Python

... '# Python Function to Check for Duplicate Characters in a String\n\n``python\ndef
has_duplicate_characters(string):\n    """\n    Checks if a string contains any duplicate
characters.\n    \n    Args:\n        string (str): The input string to check for duplicates\n
\n    Returns:\n        bool: True if the string has duplicate characters, False otherwise\n
"""\n    # Create an empty set to keep track of seen characters\n    seen_chars = set()\n
\n    # Iterate through each character in the string\n    for char in string:\n        # If
we've seen this character before, there's a duplicate\n        if char in seen_chars:\n
```

```
002_system_prompt.ipynb - 03
002_system_prompt.ipynb > messages = []
Generate + Code + Markdown | Run All | Restart | Clear All Outputs | Jupyter Variables | Outline ...
Python 3.9.6

messages = []

add_user_message(
    messages,
    "Write a Python function that checks a string for duplicate characters.",
)

answer = chat(messages, system="You are a Python engineer who writes very concise code")

answer

[33] ✓ 2.1s Python

... ```python\ndef has_duplicates(s):\n    return len(set(s)) < len(s)\n```\n\nThis function
returns `True` if the string contains any duplicate characters, and `False` otherwise. It
works by comparing the length of the string with the length of its set (which removes
duplicates).'
```

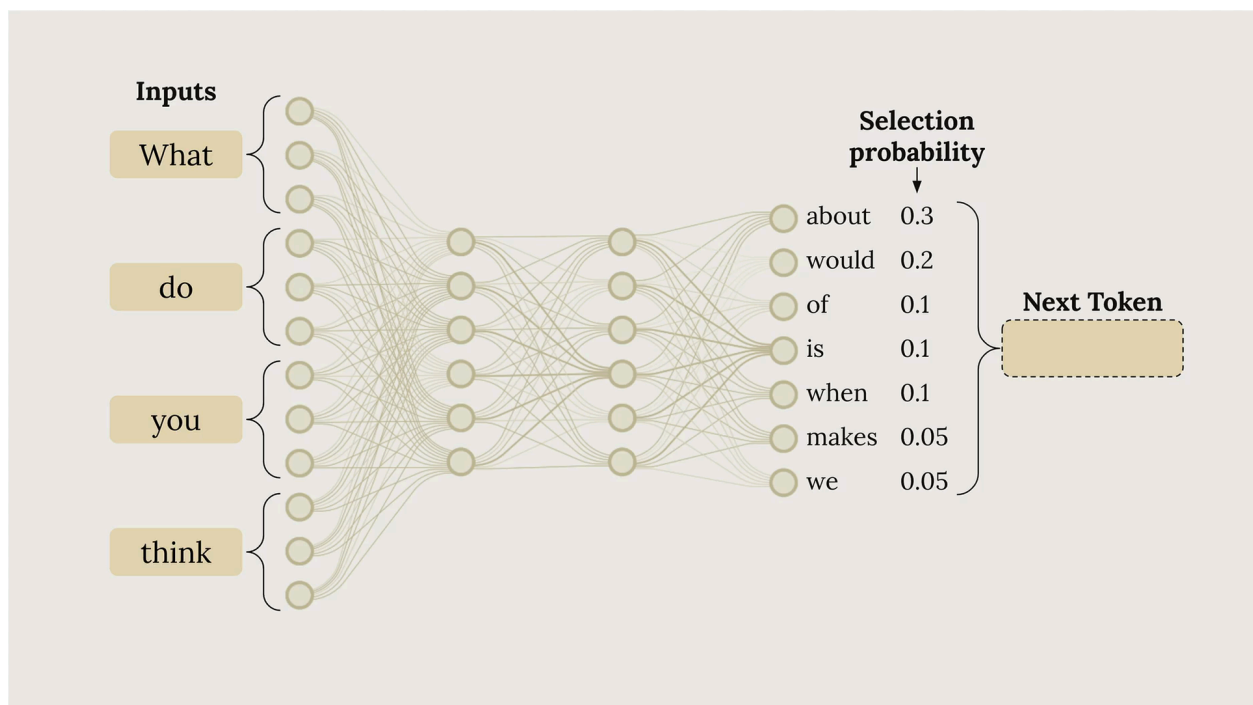
Temperature

Temperature is a powerful parameter that controls how creative or deterministic Claude's responses will be. Understanding how to use it effectively can dramatically improve your AI applications.

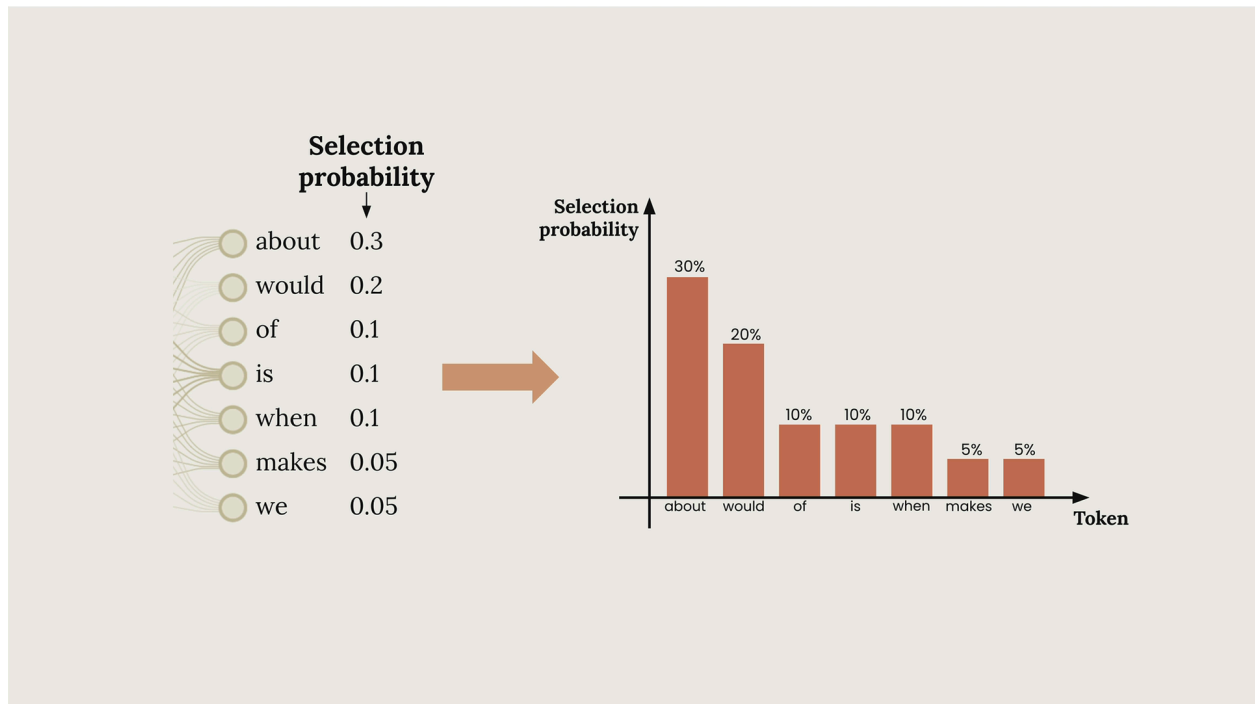
How Claude Generates Text

Before diving into temperature, it's helpful to understand Claude's text generation process. When you send Claude a prompt like "What do you think?", it goes through three phases:

- **Tokenization:** Breaking your input into smaller chunks
- **Prediction:** Calculating probabilities for possible next tokens
- **Sampling:** Selecting a token based on those probabilities



In the diagram above, you can see how Claude might assign different probabilities to potential next tokens. The word "about" has a 30% chance, "would" has 20%, and so on. This process repeats for each token until the response is complete.



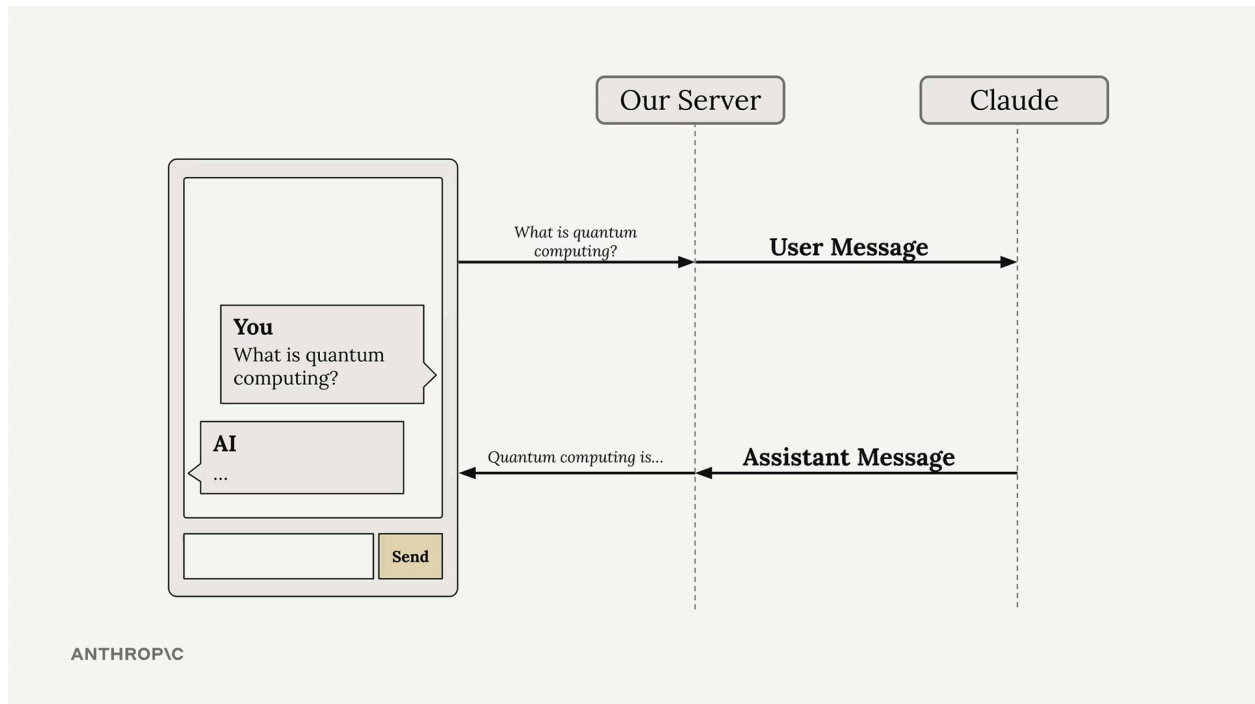
What Temperature Does

Temperature is a decimal value between 0 and 1 that directly influences these token selection probabilities. Think of it as a creativity dial:

- **Low temperature (near 0):** Makes the highest probability token much more likely to be selected
- **High temperature (near 1):** Distributes probability more evenly across all possible tokens

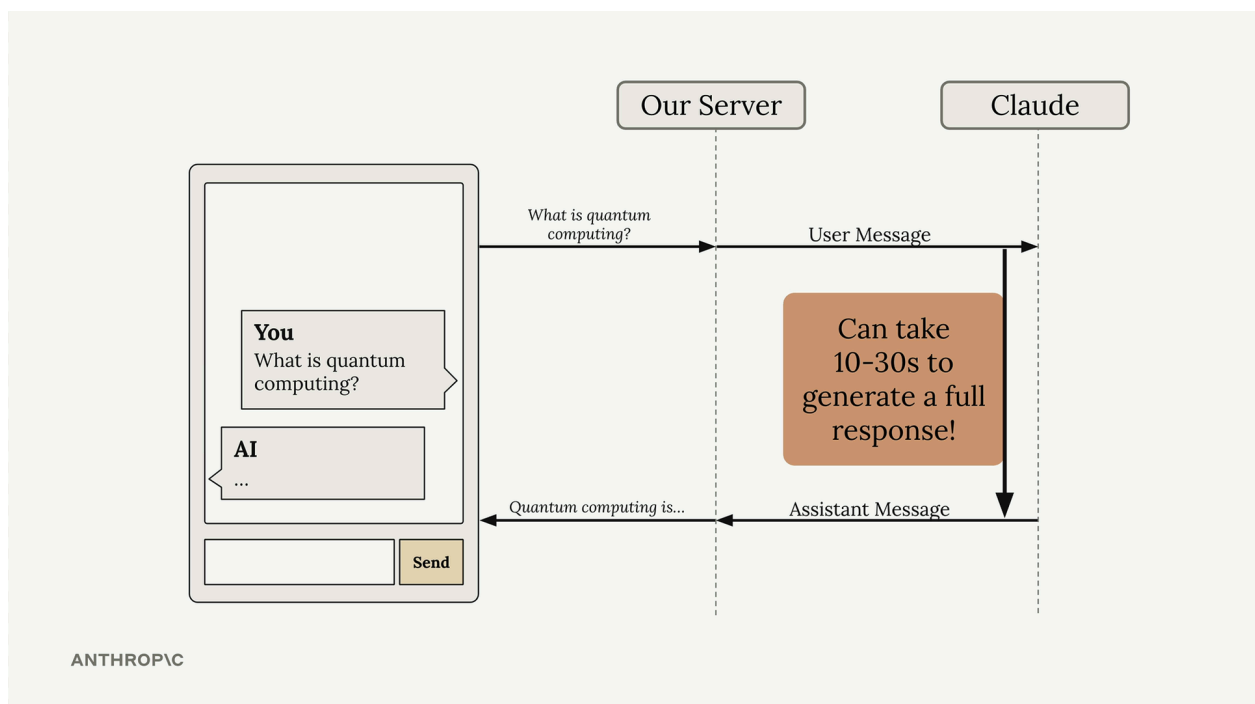
Response Streaming

When building chat applications with Claude, there's a significant user experience challenge: responses can take 10-30 seconds to generate, leaving users staring at a loading spinner. The solution is response streaming, which lets users see text appear chunk by chunk as Claude generates it, creating a much more responsive feel.



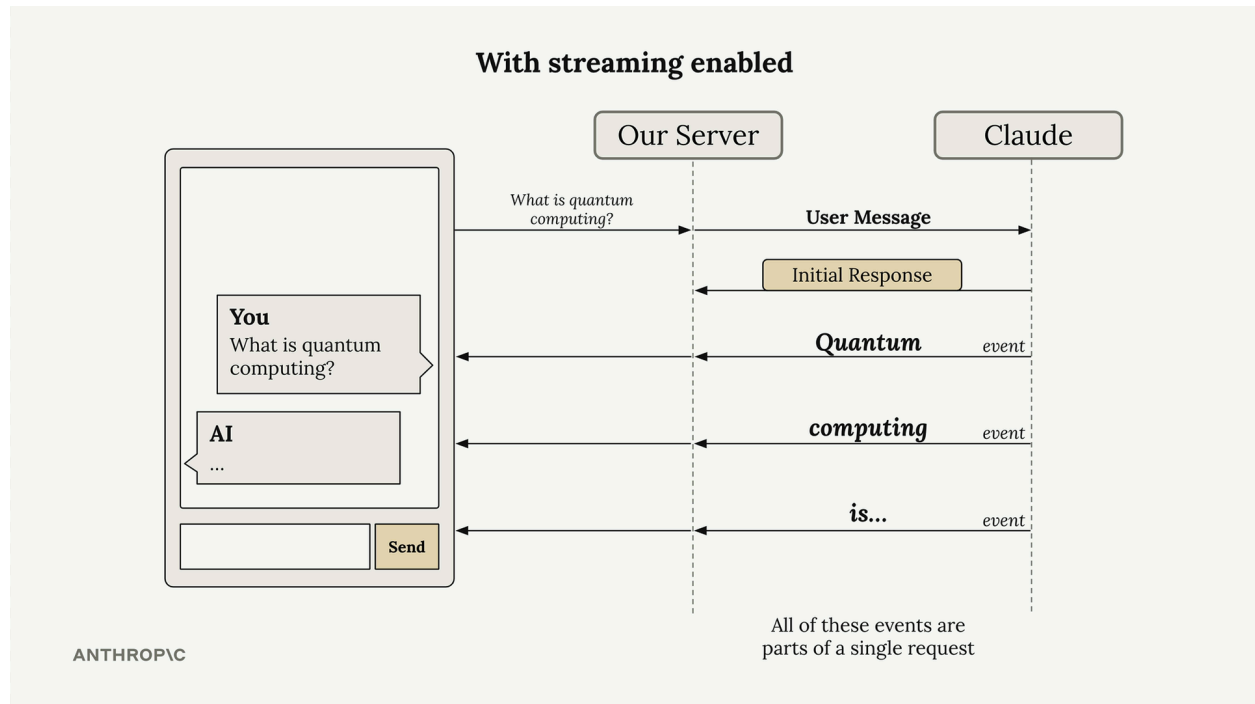
The Problem with Standard Responses

In a typical chat setup, your server sends a user message to Claude and waits for the complete response before sending anything back to the client. This creates an awkward delay where users have no feedback that anything is happening.

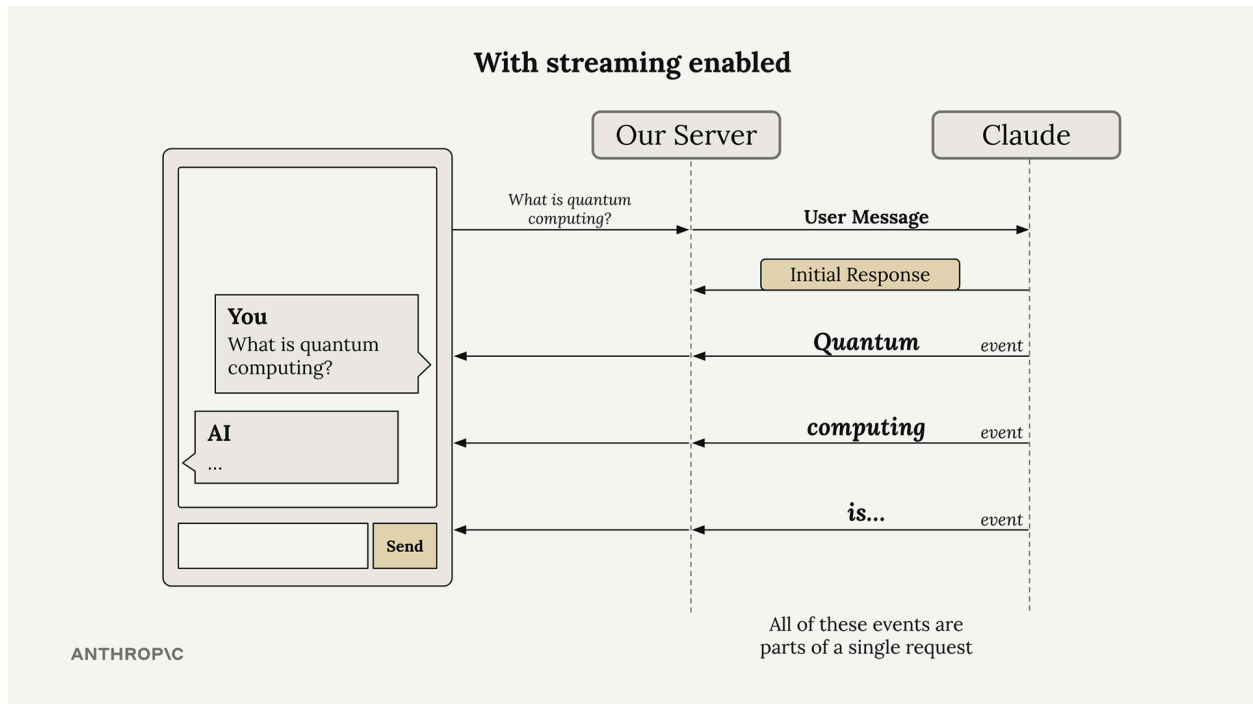


How Streaming Works

With streaming enabled, Claude immediately sends back an initial response indicating it has received your request and is starting to generate text. Then you receive a series of events, each containing a small piece of the overall response.



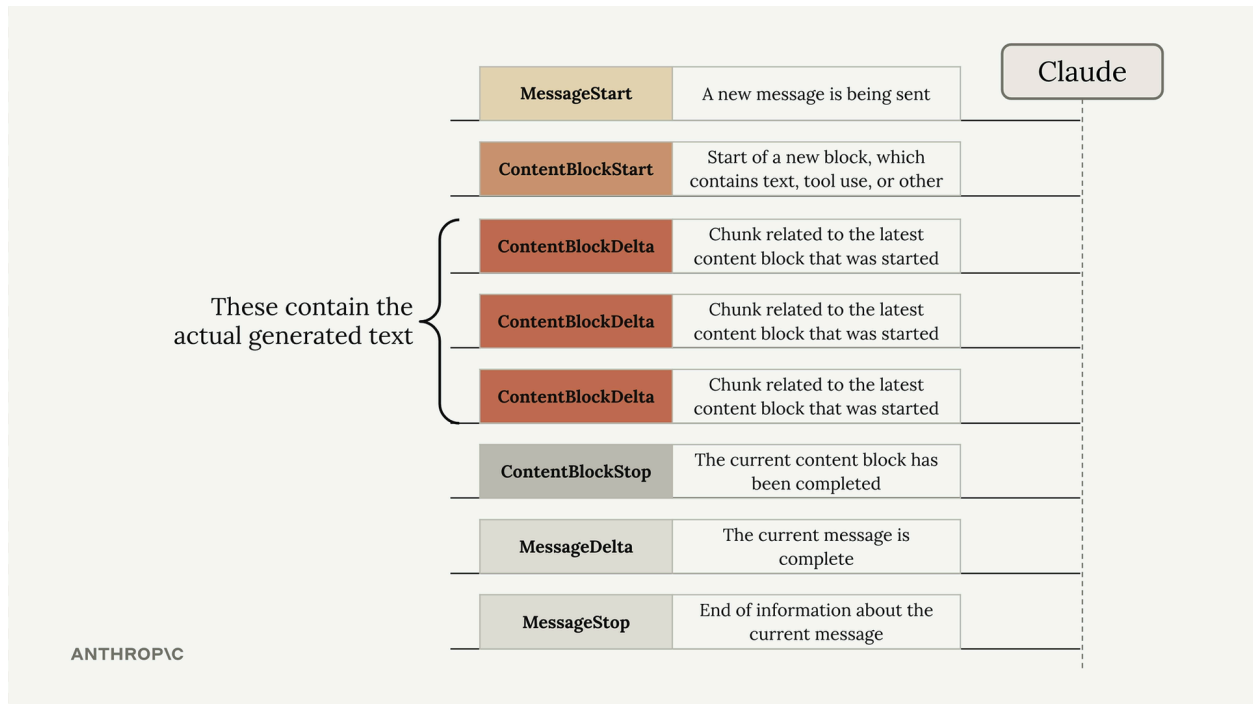
Your server can forward these text chunks to your client application as they arrive, allowing users to see the response building up word by word. All of these events are part of a single request to Claude.



Understanding Stream Events

When you enable streaming, Claude sends back several types of events:

- **MessageStart** - A new message is being sent
- **ContentBlockStart** - Start of a new block containing text, tool use, or other content
- **ContentBlockDelta** - Chunks of the actual generated text
- **ContentBlockStop** - The current content block has been completed
- **MessageDelta** - The current message is complete
- **MessageStop** - End of information about the current message



The `ContentBlockDelta` events contain the actual generated text that you'll want to display to users.

Basic Streaming Implementation

To enable streaming, add `stream=True` to your `messages.create` call:

```
messages = []
```

```
add_user_message(messages, "Write a 1 sentence description of a fake database")
```

```
stream = client.messages.create(
```

```
    model=model,
```

```
    max_tokens=1000,
```

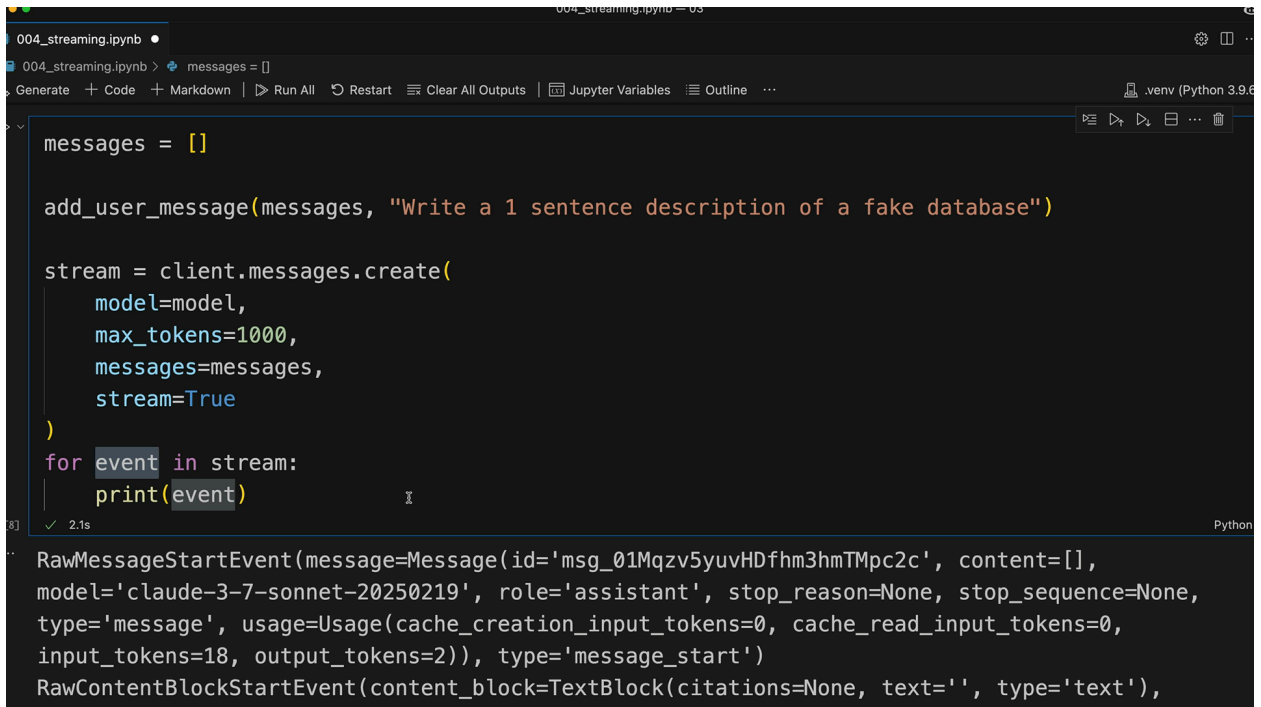
```
    messages=messages,
```

```
    stream=True
```

```
)
```

```
for event in stream:
```

```
    print(event)
```



```
004_streaming.ipynb •
004_streaming.ipynb > messages = []
Generate + Code + Markdown | Run All Restart Clear All Outputs | Jupyter Variables Outline ...
.venv (Python 3.9.6)

messages = []

add_user_message(messages, "Write a 1 sentence description of a fake database")

stream = client.messages.create(
    model=model,
    max_tokens=1000,
    messages=messages,
    stream=True
)
for event in stream:
    print(event)
```

RawMessageStartEvent(message=Message(id='msg_01Mqzv5yuvHDfhm3hmTMpc2c', content=[], model='claude-3-7-sonnet-20250219', role='assistant', stop_reason=None, stop_sequence=None, type='message', usage=Usage(cache_creation_input_tokens=0, cache_read_input_tokens=0, input_tokens=18, output_tokens=2)), type='message_start')

RawContentBlockStartEvent(content_block=TextBlock(citations=None, text='', type='text'),

Simplified Text Streaming

Rather than manually parsing events, you can use the SDK's simplified streaming interface that extracts just the text content:

```
with client.messages.stream(
```

```
    model=model,
```

```
    max_tokens=1000,
```

```
    messages=messages
```

```
) as stream:
```

```
    for text in stream.text_stream:
```

```
        print(text, end="")
```

This approach automatically filters out everything except the actual text content, which is usually what you need for displaying responses to users.

Getting the Complete Message

While streaming individual chunks is great for user experience, you often need the complete message for storage or further processing. After streaming completes, you can get the assembled final message:

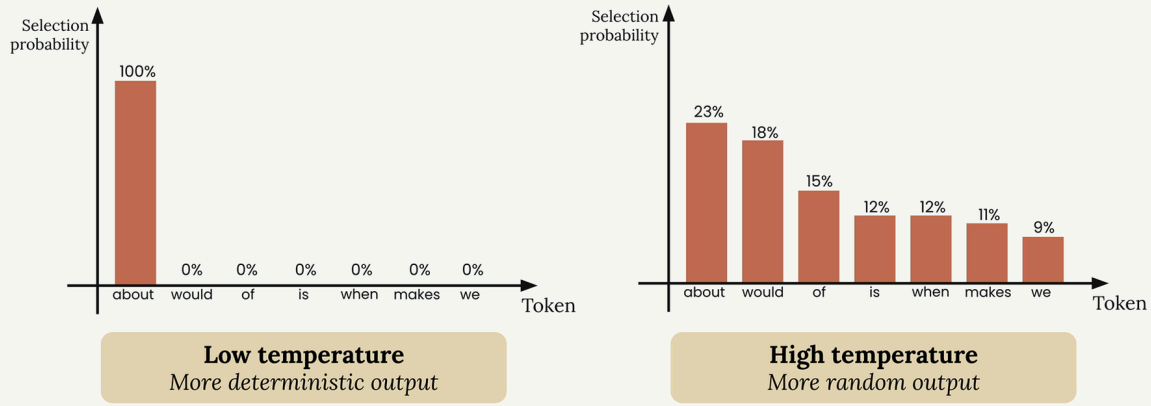
```
with client.messages.stream(
    model=model,
    max_tokens=1000,
    messages=messages
) as stream:

    for text in stream.text_stream:
        # Send each chunk to your client
        pass

    # Get the complete message for database storage
    final_message = stream.get_final_message()
```

This gives you the best of both worlds: real-time streaming for users and a complete message object for your application logic.

Temperature changes how likely each token is to be selected

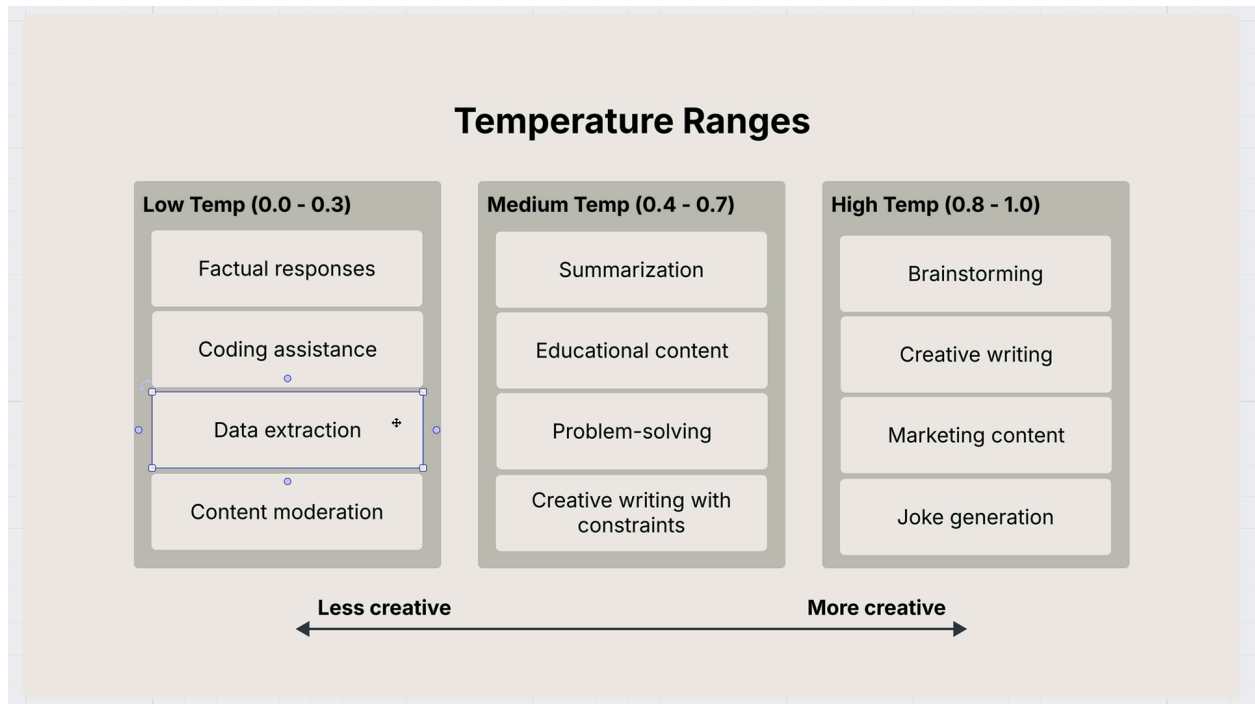


ANTHROPIC

At temperature 0, Claude becomes deterministic - it will always pick the most probable token. At temperature 1, lower-probability tokens have a much better chance of being selected, leading to more creative and varied outputs.

Temperature Ranges and Use Cases

Different tasks call for different temperature settings:



Low Temperature (0.0 - 0.3)

- Factual responses
- Coding assistance
- Data extraction
- Content moderation

Medium Temperature (0.4 - 0.7)

- Summarization
- Educational content
- Problem-solving
- Creative writing with constraints

High Temperature (0.8 - 1.0)

- Brainstorming
- Creative writing
- Marketing content
- Joke generation

Setting Temperature in Code

By default, Claude's temperature is set to 1.0, which means maximum creativity. You can override this by adding temperature to your inference configuration:

```
def chat(messages, system=None, temperature=1.0):
    params = {
        "modelId": model_id,
        "messages": messages,
        "inferenceConfig": {"temperature": temperature}
    }

    if system:
        params["system"] = [{"text": system}]

    response = client.converse(**params)
    return response["output"][0]["message"][0]["content"][0]["text"]
```

Temperature in Practice

Here's a practical example using movie idea generation. With temperature set to the default (1.0), you might get creative responses like:

"A reclusive origami master discovers her intricate paper creations come to life at night, leading her on a magical journey to save their miniature world from a mysterious shadow creature threatening to unfold their existence."

But when you set temperature to 0.0 for the same prompt, you'll consistently get more predictable responses:

"A time-traveling archaeologist must prevent ancient artifacts from being stolen by a tech billionaire who's using them to build a doomsday device that harnesses their forgotten power."

Running the low-temperature version multiple times will produce very similar responses, often with repeated themes like "time-traveling historian" or "time-traveling archaeologist."

Key Takeaways

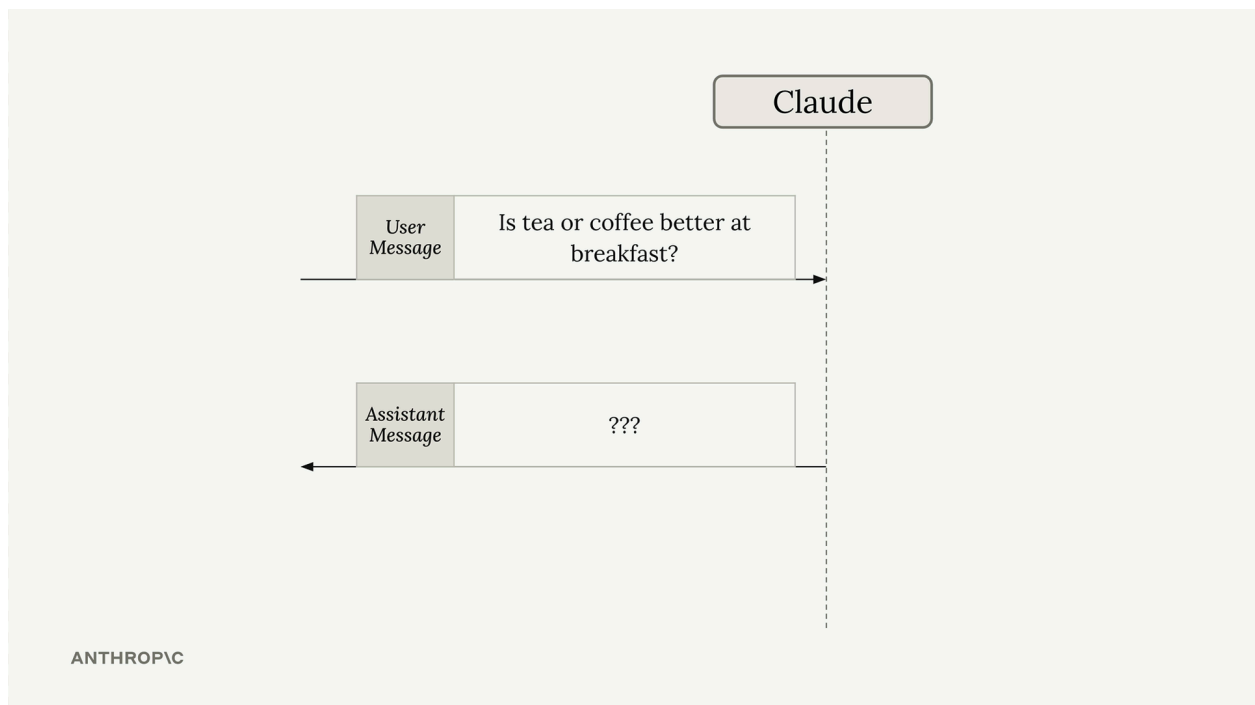
Temperature gives you direct control over Claude's creativity level. Use lower temperatures when you need consistent, factual responses, and higher temperatures when you want creative, varied outputs. The default temperature of 1.0 maximizes creativity, so consider lowering it for tasks requiring precision and consistency.

Controlling the output

Beyond crafting better prompts, there are two powerful techniques for controlling Claude's output: prefilled assistant messages and stop sequences. These methods give you precise control over how Claude responds and when it stops generating text.

Prefilled Assistant Messages

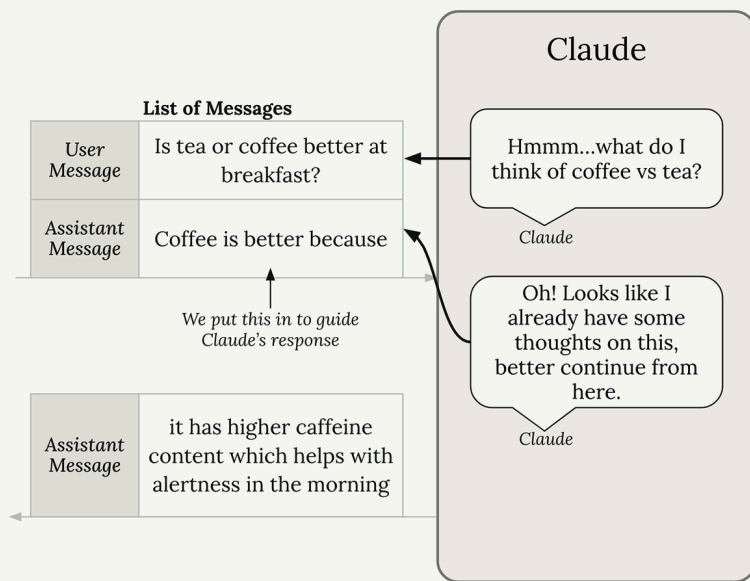
Message prefilling lets you provide the beginning of Claude's response, which strongly influences the direction of its answer. Instead of letting Claude decide how to start its response, you give it a specific opening that steers the conversation.



Here's how it works: you build your normal list of messages with the user's question, but then add an assistant message at the end containing the start of the response you want. When Claude processes this, it sees the assistant message and thinks "I've already started responding to this question, so I should continue from where I left off."

Message Prefilling

- Provide the *start* of an assistant message as your last message
- Claude will continue the response from there
- This will greatly steer Claude's response



ANTHROPIC

For example, if you ask "Is tea or coffee better at breakfast?" and prefill with "Coffee is better because", Claude will continue from that point and build a response supporting coffee. The key insight is that Claude will pick up exactly where your prefilled text ends - it won't repeat what you've written.

Let's see this in practice:

```
messages = []
add_user_message(messages, "Is coffee or tea better for breakfast?")
add_assistant_message(messages, "Coffee is better because")

chat(messages)
```

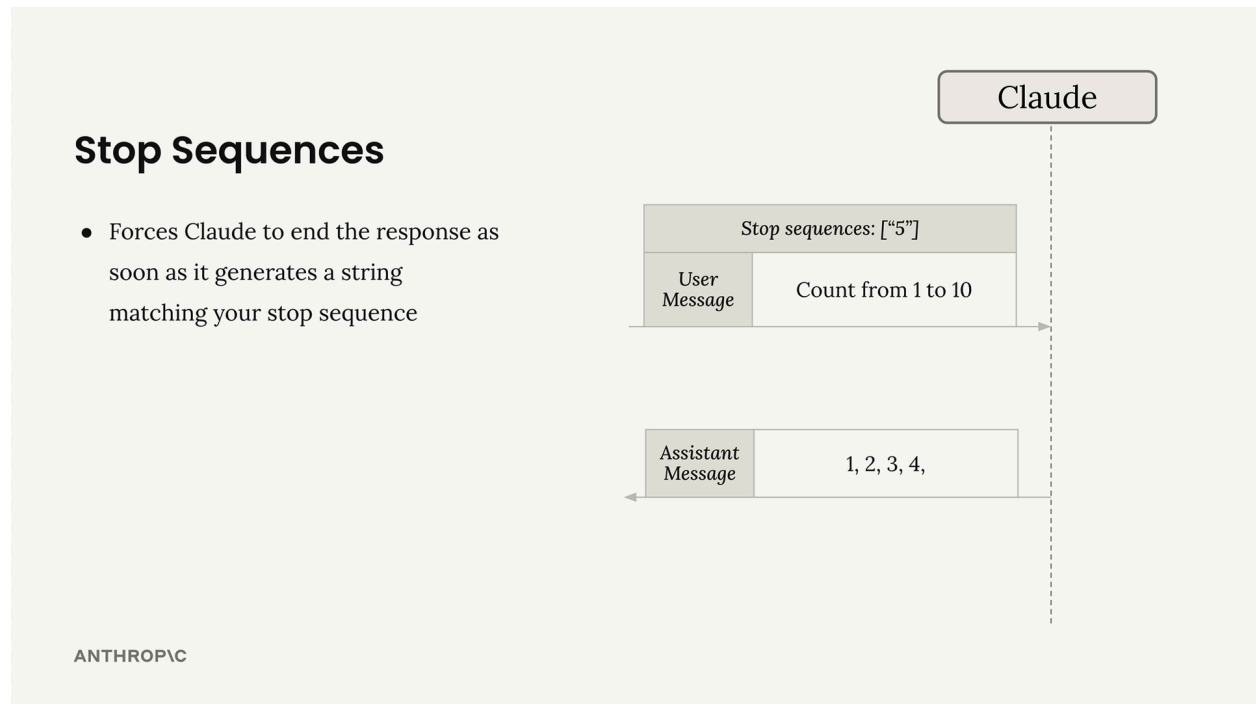
This returns something like "it has more caffeine." Notice that Claude continues directly from your prefilled text, so you'll need to combine both parts to get the complete response: "Coffee is better because it has more caffeine."

You can steer Claude in any direction by changing your prefilled text:

- "Tea is better because" - pushes toward tea
- "They are the same because" - creates a neutral response

Stop Sequences

Stop sequences force Claude to end its response immediately when it generates specific text. This is useful when you want to truncate output at a particular point or prevent Claude from continuing past a certain marker.



The concept is straightforward: you provide a list of strings, and as soon as Claude generates any of those strings, it stops and returns whatever it has generated so far. The stop sequence itself is not included in the response.

To use stop sequences, you need to modify your chat function to accept them as a parameter:

```
def chat(messages, system=None, temperature=1.0, stop_sequences=[]):
    params = {
        "modelId": model_id,
        "messages": messages,
        "inferenceConfig": {
            "temperature": temperature,
            "stopSequences": stop_sequences
        },
    }
```

Here's a practical example:

```
messages = []
add_user_message(messages, "Count from 1 to 10")

chat(messages, stop_sequences=["5"])
```

This returns "1, 2, 3, 4," and stops before including the "5". You can specify multiple stop sequences, and Claude will stop at whichever one it encounters first:

```
chat(messages, stop_sequences=["5", "3, 4"])
```

Stop sequences are particularly useful for:

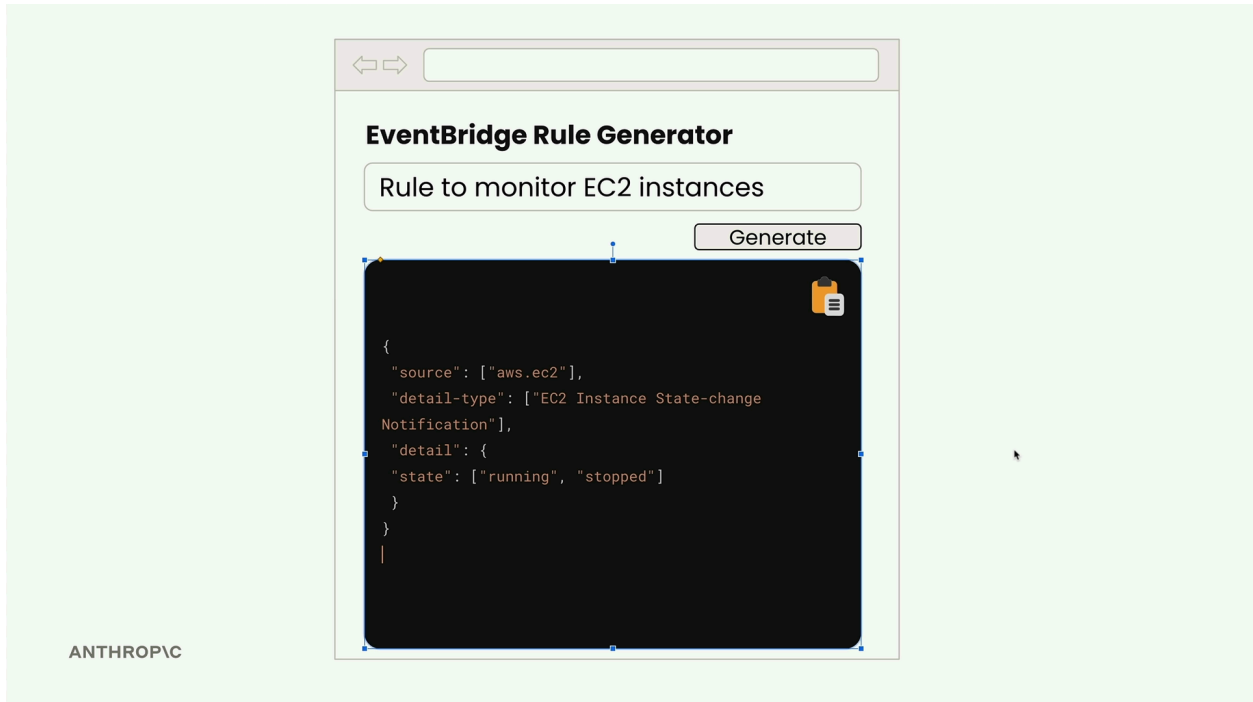
- Controlling the length of responses
- Stopping at natural breakpoints in structured output
- Preventing Claude from continuing past specific markers or delimiters

Both techniques give you fine-grained control over Claude's behavior, allowing you to shape responses in ways that simple prompting alone cannot achieve.

Structured data

When you need Claude to generate structured data like JSON, Python code, or bulleted lists, you'll often run into a common problem: Claude wants to be helpful and add explanatory text around your content. While this is usually great, sometimes you need just the raw data with nothing else.

Consider building a web app that generates AWS EventBridge rules. Users enter a description, click generate, and expect to see clean JSON they can immediately copy and use. If Claude returns the JSON wrapped in markdown code blocks with explanatory text, users can't simply copy the entire response - they have to manually select just the JSON portion.



The Problem with Default Responses

By default, when you ask Claude to generate JSON, you might get something like this:

```
```json
{
 "source": ["aws.ec2"],
 "detail-type": ["EC2 Instance State-change Notification"],
 "detail": {
 "state": ["running"]
 }
}
...
```
```

This rule captures EC2 instance state changes when instances start running.

The JSON is correct, but it's wrapped in markdown formatting and includes explanatory text. For a web app where users need to copy the raw JSON, this creates friction in the user experience.

The Solution: Assistant Message Prefilling + Stop Sequences

You can combine assistant message prefilling with stop sequences to get exactly the content you want. Here's how it works:

```
messages = []
```

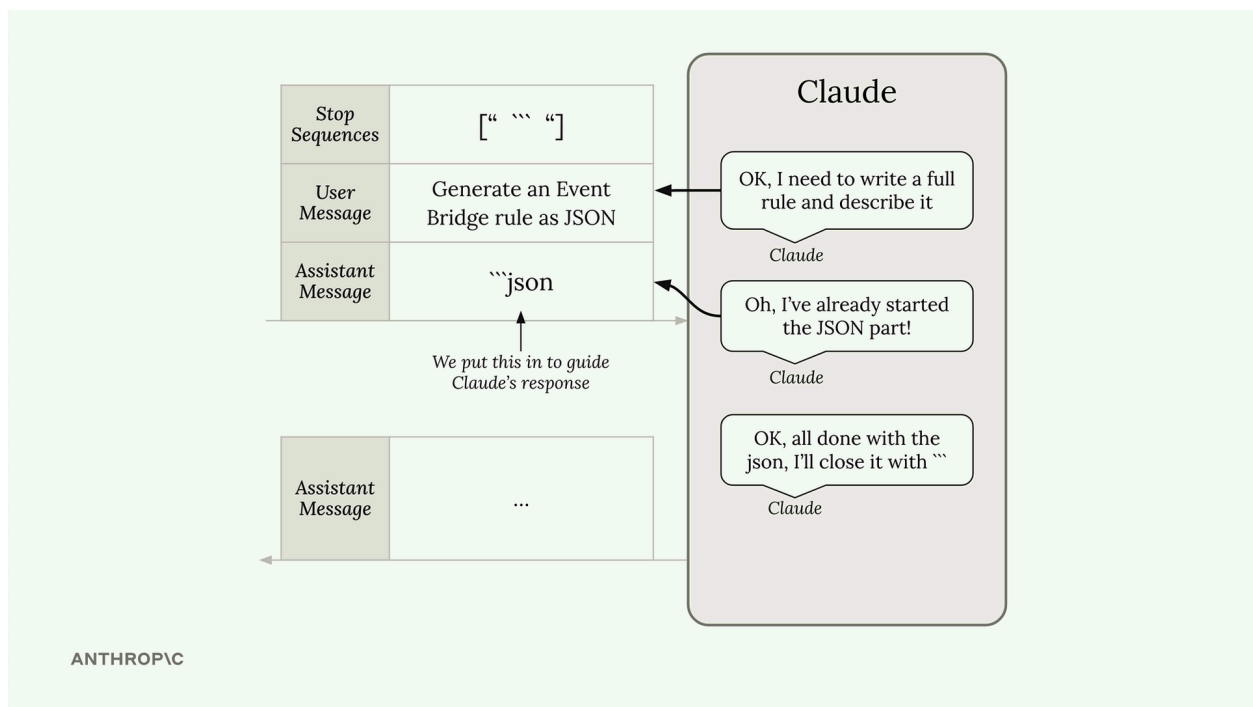
```
add_user_message(messages, "Generate a very short event bridge rule as json")
```

```
add_assistant_message(messages, "```json")
```

```
text = chat(messages, stop_sequences=["```"])
```

This technique works by:

1. The user message tells Claude what to generate
2. The prefilled assistant message makes Claude think it already started a markdown code block
3. Claude continues by writing just the JSON content
4. When Claude tries to close the code block with `````, the stop sequence immediately ends generation



The result is clean JSON with no extra formatting:

```
{  
  "source": ["aws.ec2"],  
  "detail-type": ["EC2 Instance State-change Notification"],  
  "detail": {  
    "state": ["running"]  
  }  
}
```

```
}  
}
```

Processing the Response

You might notice some extra newline characters in the response. These are easy to handle:

```
import json  
  
# Clean up and parse the JSON  
clean_json = json.loads(text.strip())
```

Beyond JSON

This technique isn't limited to JSON generation. Use it anytime you need structured data without commentary:

- Python code snippets
- Bulleted lists
- CSV data
- Any formatted content where you want just the content, not explanations

The key is identifying what Claude naturally wants to wrap your content in, then using that as your prefill and stop sequence. For code, it's usually markdown code blocks. For lists, it might be different formatting markers.

This approach gives you precise control over Claude's output format, making it much easier to integrate AI-generated content into applications where clean, structured data is essential.

Structured data exercise

Exercise!

- Use message prefilling and stop sequences *only* to get three different commands in a single response
- There shouldn't be any comments or explanation
- Hint: message prefilling isn't limited to just characters like ``

```
messages = []

prompt = """
Generate three different sample AWS CLI commands. Each should be very short.
"""

add_user_message(messages, prompt)

text = chat(messages)
text.strip()
```

```
messages = []

prompt = """
Generate three different sample AWS CLI commands. Each should be very short.
"""

add_user_message(messages, prompt)
add_assistant_message(messages, "")

text = chat(messages, stop_sequences=[""])
text.strip()
```

```
'bash\naws s3 ls\naws ec2 describe-instances\naws lambda list-functions'
```

```
from IPython.display import Markdown
```


- Hint: message prefilling isn't limited to just characters like ``

```

messages = []

prompt = """
Generate three different sample AWS CLI commands. Each should be very short.
"""

add_user_message(messages, prompt)
add_assistant_message(messages, "``bash")

text = chat(messages, stop_sequences=["``"])
text.strip()

```

✓ 9.7s

Python

```
'bash\naws s3 ls\naws ec2 describe-instances\naws lambda list-functions'
```

- There shouldn't be any comments or explanation
- Hint: message prefilling isn't limited to just characters like ``

```
: sample AWS CLI commands. Each should be very short.
```

```

s, prompt)
ssages, "Here are all three commands in a single block without any comments:\n``bash")

op_sequences=["``"])

```

✓ 0.7s

Python

```
'aws s3 ls'
```